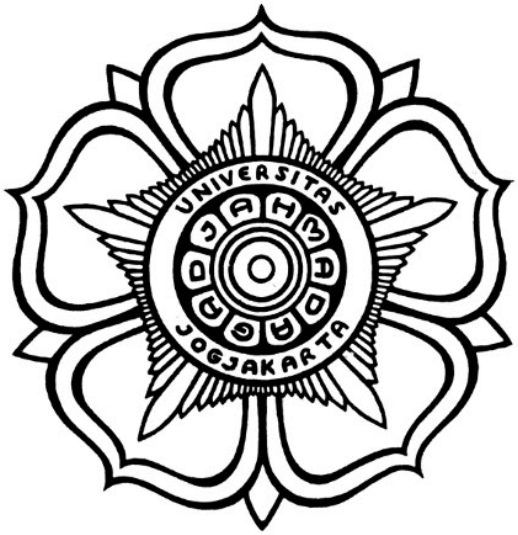
«JUDUL SKRIPSI»

SKRIPSI



THE SUSTAINABLE DEVELOPMENT GOALS
Industry, Innovation and Infrastructure
Affordable and Clean Energy
Climate Action

Disusun oleh:

Muhammad Shafa Adhitiya
22/496402/TK/54378

PROGRAM SARJANA PROGRAM STUDI TEKNIK ELEKTRO
DEPARTEMEN TEKNIK ELEKTRO DAN TEKNOLOGI INFORMASI
FAKULTAS TEKNIK UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

HALAMAN PENGESAHAN

«JUDUL SKRIPSI»

SKRIPSI

Diajukan Sebagai Salah Satu Syarat untuk Memperoleh
Gelar Sarjana Teknik
pada Departemen Teknik Elektro dan Teknologi Informasi
Fakultas Teknik
Universitas Gadjah Mada

Disusun oleh:

Muhammad Shafa Adhitiya
22/496402/TK/54378

Telah disetujui dan disahkan

Pada tanggal . . . . . .

Dosen Pembimbing I	Dosen Pembimbing II
Ir. Agus Bejo, S.T., M.Eng., D.Eng., IPM.	**Dosen Pembimbing 2, S.T., M.Eng., PhD.**
NIP 198001012015041002	**NIP 198211082010121002**

PERNYATAAN BEBAS PLAGIASI

Saya yang bertanda tangan di bawah ini :

Nama	:
NIM	:
Tahun terdaftar	:
Program	: Sarjana
Program Studi	:
Fakultas	: Teknik Universitas Gadjah Mada

Menyatakan bahwa dalam dokumen ilmiah Skripsi ini tidak terdapat bagian dari karya ilmiah lain yang telah diajukan untuk memperoleh gelar akademik di suatu lembaga Pendidikan Tinggi, dan juga tidak terdapat karya atau pendapat yang pernah ditulis atau diterbitkan oleh orang/lembaga lain, kecuali yang secara tertulis disitasi dalam dokumen ini dan disebutkan sumbernya secara lengkap dalam daftar pustaka.

Dengan demikian saya menyatakan bahwa dokumen ilmiah ini bebas dari unsur-unsur plagiasi dan apabila dokumen ilmiah Skripsi ini di kemudian hari terbukti merupakan plagiasi dari hasil karya penulis lain dan/atau dengan sengaja mengajukan karya atau pendapat yang merupakan hasil karya penulis lain, maka penulis bersedia menerima sanksi akademik dan/atau sanksi hukum yang berlaku.

Yogyakarta, tanggal-bulan-tahun

Materai Rp10.000

(Tanda tangan)

Nama Mahasiswa
NIM

HALAMAN PERSEMBAHAN

Tuliskan kepada siapa skripsi ini dipersembahkan!

contoh

KATA PENGANTAR

Contoh: Puji syukur ke hadirat Allah SWT atas limpahan rahmat, karunia, serta petunjuk-Nya sehingga tugas akhir berupa penyusunan skripsi ini telah terselesaikan dengan baik. Dalam hal penyusunan tugas akhir ini penulis telah banyak mendapatkan arahan, bantuan, serta dukungan dari berbagai pihak. Oleh karena itu pada kesempatan ini penulis mengucapkan terima kasih kepada:

1. Bapak Prof. Ir. Hanung Adi Nugroho, S.T., M.Eng., Ph.D., IPM., SMIEEE. selaku Ketua Departemen Teknik Elektro dan Teknologi Informasi Fakultas Teknik Universitas Gadjah Mada.
2. Bapak Dr. Iswandi, S.T., M.Eng./Dr. Ahmad Nasikun, S.T., M.Sc./Syukron Abu Ishaq Alfarozi, S.T., Ph.D. (**pilih yang sesuai**) selaku Ketua Program Studi Teknik Elektro/Teknologi Informasi/Teknik Biomedis (**pilih yang sesuai**).
3. Bapak · · · · · · · · · selaku dosen pembimbing pertama · · · · · · · · · .
4. Ibu · · · · · · · · · selaku dosen pembimbing kedua · · · · · · · · · .
5. <isi dengan nama orang lainnya>

Akhir kata penulis berharap semoga skripsi ini dapat memberikan manfaat bagi kita semua, aamiin.

Yogyakarta, <tanggal harus sebelum tanggal pendadaran>

Penulis

DAFTAR ISI

HALAMAN PENGESAHAN ii
PERNYATAAN BEBAS PLAGIASI iii
HALAMAN PERSEMBAHAN iv
KATA PENGANTAR v
DAFTAR ISI vi
DAFTAR TABEL ix
DAFTAR GAMBAR x
DAFTAR SINGKATAN xi
INTISARI xiii
ABSTRACT xiv
BAB I Pendahuluan 1
1.1 Latar Belakang 1
1.2 Rumusan Masalah 2
1.3 Tujuan Penelitian 4
1.4 Batasan Penelitian 7
1.5 Manfaat Penelitian 7
1.6 Sistematika Penulisan 8
BAB II Tinjauan Pustaka dan Dasar Teori 9
2.1 Tinjauan Pustaka 9
2.2 Dasar Teori 9
2.2.1 Pengenalan Aplikasi Permainan 9
2.2.2 Dasar Teori Lainnya 10
2.3 Analisis Perbandingan Metode 10
2.4 Pertanyaan Tugas Akhir (Jika Perlu) 10
BAB III Metode Penelitian 11
3.1 Alat dan Bahan Tugas Akhir 11
3.1.1 Alat Tugas Akhir (*Hardware*) 11
3.1.2 Bahan Tugas Akhir 11
3.2 Metode yang Digunakan 12
3.3 Alur Tugas Akhir 12
3.4 Struktur Hierarki *Soft-core* mriscv 16
3.4.1 Hierarki Modul RTL Verilog 16
3.4.2 Struktur Internal `mriscvcore` 18
3.4.3 Bus AXI4-Lite dan Modul Periferal 19
3.4.4 Rancangan Modul Periferal UART dan I2C 21
3.5 Metode Analisis Hasil Akhir 23

3.5.1 Analisis Fungsional 23
3.5.2 Analisis Kuantitatif 23
BAB IV Hasil dan Pembahasan 25
4.1 Keberhasilan Sintesis dan Implementasi di FPGA 25
4.2 Mekanisme *Program Loading* dan Pengembangan *Firmware* 25
4.3 Hasil Pengujian Fungsional pada *Hardware* 27
4.4 Analisis dan Verifikasi *Source Code* RTL 28
4.4.1 Modifikasi `SP32B1024.v` 29
4.4.2 Modifikasi *Latch* pada `axi4_interconnect.v` 29
4.4.3 *Tri-state* pada `axi4_interconnect.v`: Ditangani Otomatis oleh Yosys (TRIBUF) 30
4.4.4 *Negedge* pada `AXI_SP32B1024.v`: Disengaja, *Patch* Naif Terbukti Merusak 30
4.4.5 *Tri-state* pada `mriscvcore.v`/`ALU.v`: Temuan Utama yang Merusak Seluruh *Write-back Path* 31
4.4.6 Parameter `addr_mask`/`addr_use` Tidak Ter-*override*: Ekspansi RAM dan Relokasi Periferal Sempat Mengalami *Silent Failure* 33
4.4.7 Multi-*clock* ke Single-*clock* pada `spi_axi_master.v` 34
4.5 *Maximum Operating Frequency* (*Fmax*) Hasil *Place-and-Route* dan *Resource Utilization* 34
4.5.1 Eksperimen A: *Repeatability* 34
4.5.2 Eksperimen B: Sensitivitas *Seed* 35
4.6 Penambahan Periferal UART dan I2C *Controller* 36
4.6.1 Integrasi ke Bus AXI4-Lite 37
4.6.2 Implementasi UART 37
4.6.3 Implementasi I2C *Controller* 39
4.6.4 Hasil Verifikasi Simulasi dan *Hardware* 40
4.6.5 Temuan: *Hang* pada `uart_puts()` Disebabkan *Linker Script* 42
4.6.6 *Resource Utilization* dan *Fmax* Setelah Penambahan Periferal 43
4.7 Perbandingan dengan Penelitian Terdahulu 45
BAB V Tambahan (Opsional) 47
BAB VI Kesimpulan dan Saran 48
6.1 Kesimpulan 48
6.2 Saran 48
DAFTAR PUSTAKA 49
LAMPIRAN L-1
L.1 Isi Lampiran L-1
L.2 Panduan Latex L-2

L.2.1 Syntax Dasar L-2
L.2.1.1 Penggunaan Sitasi L-2
L.2.1.2 Penulisan Gambar L-2
L.2.1.3 Penulisan Tabel L-2
L.2.1.4 Penulisan formula..... L-2
L.2.1.5 Contoh list..... L-3
L.2.2 Blok Beda Halaman..... L-3
L.2.2.1 Membuat algoritma terpisah L-3
L.2.2.2 Membuat tabel terpisah..... L-3
L.2.2.3 Menulis formula terpisah halaman..... L-4
L.3 Format Penulisan Referensi L-6
L.3.1 Book L-6
L.3.2 Handbook..... L-8
L.4 Contoh Source Code L-10
L.4.1 Sample algorithm L-10
L.4.2 Sample Python code L-11
L.4.3 Sample Matlab code L-12

DAFTAR TABEL

Tabel 3.1 *Address Map* Bus AXI4-Lite 20
Tabel 3.2 *Register Map* Modul UART dan I2C 22

Tabel 4.1 Ringkasan Pemeriksaan Modifikasi RTL 28
Tabel 4.2 Hasil Eksperimen *Repeatability* Sintesis ($n = 5$ per konfigurasi) 35
Tabel 4.3 *Resource Utilization* FPGA (Xilinx Artix-7 XC7A35T) 35
Tabel 4.4 Sensitivitas *Fmax* terhadap Nilai *Seed* `nextpnr-xilinx` (RAM 32 KB, 50 MHz) 36
Tabel 4.5 Ringkasan Verifikasi Modul UART dan I2C 41
Tabel 4.6 *Resource Utilization* Modul UART dan I2C (Sintesis *Standalone*) ... 43
Tabel 4.7 *Resource Utilization* Sebelum dan Sesudah Penambahan Periferal (RAM 32 KB, 50 MHz) 44
Tabel 4.8 Perbandingan Karakteristik Implementasi dengan `mriscv_vivado` 45

Tabel L.1 Tabel ini L-2
Tabel L.2 Contoh tabel panjang L-4

DAFTAR GAMBAR

Gambar 2.1 Contoh gambar 10
Gambar 3.2 Diagram Hierarki Modul RTL dan Hubungannya melalui Bus AXI4-Lite 17
Gambar 3.3 Diagram Blok Struktur Internal `mriscvcore` 19
Gambar L.1 Contoh gambar. L-2

DAFTAR SINGKATAN

[SAMPLE]

b	=	bias
$K(x_i, x_j)$	=	fungsi kernel
y	=	kelas keluaran
C	=	parameter untuk mengendalaikan besarnya pertukaran antara penalti variabel slack dengan ukuran margin
L_D	=	persamaan Lagrange dual
L_P	=	persamaan Lagrange primal
w	=	vektor bobot
x	=	vektor masukan
ANFIS	=	Adaptive Network Fuzzy Inference System
ANSI	=	American National Standards Institute
DAG	=	Directed Acyclic Graph
DDAG	=	Decision Directed Acyclic Graph
HIS	=	Hue Saturation Intensity
QP	=	Quadratic Programming
RBF	=	Radial Basis Function
RGB	=	Red Green Blue
SV	=	Support Vector
SVM	=	Support Vector Machines

INTISARI

Intisari ditulis menggunakan bahasa Indonesia dengan jarak antar baris 1 spasi dan maksimal 1 halaman. Intisari sekurang-kurangnya berisi tentang latar belakang dan tujuan penelitian, metodologi yang digunakan, hasil penelitian, kesimpulan dan implikasi, dan Kata kunci yang berhubungan dengan penelitian.

Kata Kunci ditulis maksimal 5 kata yang paling berhubungan dengan isi skripsi. Silakan mengacu pada ACM / IEEE *Computing classification* jika Anda adalah mahasiswa Sarjana TI http://www.acm.org/about/class/ atau mengacu kepada IEEE keywords http://www.ieee.org/documents/taxonomy_v101.pdf jika Anda berasal dari Prodi Sarjana TE.

Kata kunci : Kata kunci 1, Kata kunci 2, Kata kunci 3, Kata kunci 4, Kata kunci 5

Contoh Abstrak Teknik Elektro:

"Penelitian ini bertujuan untuk mengembangkan sistem pengendalian suhu ruangan dengan menggunakan microcontroller. Metodologi yang digunakan adalah desain sirkuit, implementasi sistem pengendalian, dan pengujian performa. Hasil penelitian menunjukkan bahwa sistem pengendalian suhu ruangan yang dikembangkan mampu mengendalikan suhu ruangan dengan akurasi sebesar ±0,5°C. Kesimpulan dari penelitian ini adalah sistem pengendalian suhu ruangan yang dikembangkan efektif dan efisien.

Kata kunci: microcontroller, sistem pengendalian suhu, akurasi."

Contoh Abstrak Teknik Biomedis:

"Penelitian ini bertujuan untuk mengevaluasi keefektifan prototipe alat pemantau denyut jantung berbasis elektrokardiogram (ECG) untuk pasien jantung. Metodologi yang digunakan meliputi desain dan pembuatan prototipe, pengujian dengan pasien, dan analisis data. Hasil penelitian menunjukkan bahwa prototipe alat pemantau denyut jantung berbasis ECG memiliki akurasi yang baik dan mampu memantau denyut jantung pasien secara efektif. Kesimpulan dari penelitian ini adalah prototipe alat pemantau denyut jantung berbasis ECG merupakan solusi yang efektif dan efisien untuk memantau pasien jantung.

Kata kunci: elektrokardiogram, alat pemantau denyut jantung, akurasi."

Contoh Abstrak Teknologi Informasi:

"Penelitian ini bertujuan untuk mengevaluasi keamanan dan privasi pengguna aplikasi media sosial terpopuler. Metodologi yang digunakan meliputi analisis kebijakan privasi dan pengaturan keamanan, pengujian penetrasi, dan survei pengguna. Hasil penelitian menunjukkan bahwa beberapa aplikasi media sosial memiliki kebijakan privasi yang kurang jelas dan rendahnya tingkat keamanan. Kesimpulan dari penelitian ini adalah pentingnya meningkatkan kebijakan privasi dan tingkat keamanan pada aplikasi media sosial untuk melindungi privasi dan data pengguna.

Kata kunci: media sosial, keamanan, privasi, pengguna."

ABSTRACT

Abstract ditulis italic (miring) menggunakan bahasa Inggris dengan jarak antar baris 1 spasi dan maksimal 1 halaman. Abstract adalah versi Bahasa Inggris dari intisari. Abstract dapat ditulis dalam beberapa paragraf. Baris pertama paragraph harus menjorok ke dalam sekitar 1 cm. Tidak dsarankan menggunakan mesin penerjemah melainkan tulis ulang.

Keywords : Keyword 1, Keyword 2, Keyword 3, Keyword 4, Keyword 5

BAB I

PENDAHULUAN

1.1 Latar Belakang

Sub-bab ini berisi uraian tentang latar belakang atau justifikasi ilmiah (dari *paper* dan jurnal) dari permasalahan yang akan diteliti, alasan penelitian dan penelitian yang pernah dilakukan sebelumnya terkait fenomena tersebut.

Contoh latar belakang penelitian untuk teknik elektro:

"Peningkatan konsumsi energi listrik yang terus menerus menyebabkan ketersediaan sumber energi yang semakin terbatas. Sumber energi terbarukan seperti solar dan angin menjadi solusi yang menjanjikan untuk memenuhi kebutuhan energi. Namun, kapasitas produksi dan efisiensi dari sumber energi terbarukan masih sangat tergantung pada kondisi cuaca dan geografis. Oleh karena itu, perlu adanya sistem penyimpanan energi yang efisien dan dapat memastikan ketersediaan energi listrik secara kontinu. Penelitian ini akan mengevaluasi kemampuan superkapasitor dalam menyimpan dan mengirimkan energi secara efisien dan memastikan ketersediaan energi listrik secara kontinu."

Latar belakang ini memperkenalkan masalah ketersediaan sumber energi dan peningkatan konsumsi energi listrik yang terus menerus. Ini juga memperkenalkan solusi yang menjanjikan dari sumber energi terbarukan dan menjelaskan mengapa perlu adanya sistem penyimpanan energi yang efisien. Latar belakang ini memberikan dasar yang kuat bagi perumusan masalah dan tujuan penelitian, memastikan bahwa hasil penelitian memiliki relevansi dan signifikansi bagi bidang terkait.

Contoh latar belakang penelitian untuk teknik biomedis:

"Diagnosis dan pengobatan penyakit memerlukan integrasi informasi medis yang akurat dan terkini. Alat diagnostik tradisional seperti CT scan dan MRI memiliki resolusi yang tinggi dan dapat mengidentifikasi masalah pada tingkat sel, tetapi sering memerlukan banyak waktu dan biaya. Alat deteksi dini seperti tes darah dan urin memiliki biaya rendah dan mudah digunakan, tetapi sering kurang akurat dan tidak memberikan gambaran yang jelas tentang masalah medis. Oleh karena itu, penting untuk menemukan metode baru yang memadukan keunggulan dari kedua jenis alat tersebut. Penelitian ini akan mengevaluasi kemampuan nanopartikel dalam meningkatkan akurasi dan efisiensi diagnosis medis."

Latar belakang ini memperkenalkan masalah diagnostik dan pengobatan penyakit dan mempertimbangkan pentingnya integrasi informasi medis yang akurat dan terkini. Ini juga memperkenalkan kelemahan dari alat diagnostik tradisional dan deteksi

dini dan menjelaskan mengapa penting untuk menemukan metode baru yang memadukan keunggulan dari kedua jenis alat tersebut. Latar belakang ini memberikan dasar yang kuat bagi perumusan masalah dan tujuan penelitian, memastikan bahwa hasil penelitian memiliki relevansi dan signifikansi bagi bidang terkait.

Contoh latar belakang penelitian untuk teknologi informasi:

"Dengan perkembangan teknologi informasi yang sangat pesat dalam beberapa tahun terakhir, penyimpanan data menjadi masalah yang semakin penting. Semakin banyak data yang diterima setiap hari, semakin penting bagi organisasi untuk memastikan bahwa data mereka aman dan terlindungi. Pada saat yang sama, organisasi juga membutuhkan akses cepat dan efisien ke data mereka untuk membuat keputusan yang tepat. Teknologi enkripsi kuantum baru-baru ini muncul sebagai solusi potensial untuk memenuhi kebutuhan ini, dengan menawarkan tingkat keamanan yang jauh lebih tinggi dan proses enkripsi yang lebih cepat dibandingkan dengan teknologi enkripsi konvensional. Oleh karena itu, penting untuk mengevaluasi efektivitas dan keamanan teknologi enkripsi kuantum dalam sistem penyimpanan data cloud."

Latar belakang ini memperkenalkan masalah penyimpanan data dan mempertimbangkan pentingnya keamanan data. Ini juga memperkenalkan teknologi enkripsi kuantum sebagai solusi potensial dan menjelaskan mengapa evaluasi teknologi ini penting bagi bidang teknologi informasi. Latar belakang ini memberikan dasar yang kuat bagi perumusan masalah dan tujuan penelitian, memastikan bahwa hasil penelitian memiliki relevansi dan signifikansi bagi bidang terkait.

1.2 Rumusan Masalah

Sub-bab ini berisi poin-poin yang menjelaskan masalah atau isu yang akan diteliti dalam suatu penelitian. Ini mencakup formulasi masalah yang jelas dan spesifik dan mempertimbangkan latar belakang dan deskripsi masalah yang terkait yang tertulis Dalam latar belakang pada sub-bab sebelumnya. Perumusan masalah yang baik akan membantu menentukan fokus penelitian dan memberikan dasar untuk tujuan dan hipotesis penelitian. Dalam penelitian, perumusan masalah memainkan peran penting dalam memastikan bahwa hasil penelitian berguna dan relevan untuk masalah yang diteliti.

Jika rumusan masalah lebih dari 1, maka ditulis dengan format *list*. Contoh sebagai berikut:

1. Rumusan masalah 1.
2. Rumusan masalah 2.

Contoh perumusan masalah untuk **Teknik Elektro**:

"Bagaimana memperbaiki efisiensi penghematan energi pada sistem pencahayaan rumah tangga melalui implementasi teknologi kontrol otomatis?"

Perumusan masalah ini jelas dan spesifik dan menentukan fokus penelitian pada perbaikan efisiensi penghematan energi dalam sistem pencahayaan rumah tangga dengan menggunakan teknologi kontrol otomatis. Ini juga mempertimbangkan latar belakang tentang pentingnya penghematan energi dan memberikan solusi praktis melalui implementasi teknologi. Perumusan masalah ini memberikan dasar yang kuat untuk tujuan dan hipotesis penelitian dan memastikan bahwa hasil penelitian akan berguna bagi bidang teknik elektro.

Contoh perumusan masalah untuk **Teknik Biomedis**:

"Bagaimana memperbaiki akurasi deteksi kanker payudara dengan menggunakan teknologi pemindaian ultrasonografi berbasis AI?"

Perumusan masalah ini jelas dan spesifik dan menentukan fokus penelitian pada perbaikan akurasi deteksi kanker payudara dengan menggunakan teknologi pemindaian ultrasonografi berbasis AI. Ini mempertimbangkan latar belakang tentang pentingnya deteksi dini kanker payudara dan memberikan solusi praktis melalui implementasi teknologi. Perumusan masalah ini memberikan dasar yang kuat untuk tujuan dan hipotesis penelitian dan memastikan bahwa hasil penelitian akan berguna bagi bidang teknik biomedis.

Contoh perumusan masalah untuk **Teknologi Informasi**:

"Bagaimana meningkatkan efisiensi dan keamanan sistem penyimpanan data cloud melalui implementasi teknologi enkripsi kuantum?"

Perumusan masalah ini jelas dan spesifik dan menentukan fokus penelitian pada peningkatan efisiensi dan keamanan sistem penyimpanan data cloud dengan menggunakan teknologi enkripsi kuantum. Ini mempertimbangkan latar belakang tentang pentingnya keamanan data dan memberikan solusi praktis melalui implementasi teknologi. Perumusan masalah ini memberikan dasar yang kuat untuk tujuan dan hipotesis penelitian dan memastikan bahwa hasil penelitian akan berguna bagi bidang teknologi informasi.

1.3 Tujuan Penelitian

Tujuan penelitian pada skripsi Teknik (TE, TB, TIF) adalah menentukan sasaran atau target yang ingin dicapai melalui proses penelitian. Tujuan penelitian bisa beragam sesuai dengan bidang ilmu yang dipelajari, topik penelitian, dan permasalahan yang akan dicari solusinya.

Tujuan perlu disinkronkan dengan rumusan masalah. Setiap rumusan masalah boleh memiliki lebih dari 1 tujuan. Berikut ini adalah contoh tujuan dari rumusan masalah "Bagaimanakan memperbaiki efisiensi penghematan energi pada sistem pencahayaan rumah tangga melalui implementasi teknologi kontrol otomatis?"

1. Memodelkan sistem pencahayaan rumah tangga;
2. Mendesain sistem kontrol otomatis untuk sistem pencahayaan rumah tangga.

Catatan: Tujuan penelitian dalam skripsi bidang teknik harus jelas, spesifik, terukur, dan dapat dicapai dalam jangka waktu yang ditentukan.

Berikut ini adalah contoh detail tujuan untuk masing-masing prodi di DTETI:

Contoh Tujuan Penelitian Skripsi Teknik Elektro:

Berikut adalah beberapa contoh tujuan penelitian yang sesuai dengan rumusan masalah "perbaikan efisiensi penghematan energi pada sistem pencahayaan rumah tangga melalui implementasi teknologi kontrol otomatis":

1. Menganalisis tingkat efisiensi energi pada sistem pencahayaan rumah tangga sebelum dan setelah implementasi teknologi kontrol otomatis.
2. Mengukur pengurangan biaya listrik setelah implementasi teknologi kontrol otomatis pada sistem pencahayaan rumah tangga.
3. Menunjukkan bagaimana teknologi kontrol otomatis dapat memperbaiki efisiensi penghematan energi pada sistem pencahayaan rumah tangga.
4. Meningkatkan kenyamanan dan keamanan pengguna rumah tangga melalui penggunaan teknologi kontrol otomatis pada sistem pencahayaan.
5. Menjelaskan bagaimana implementasi teknologi kontrol otomatis mempengaruhi kualitas cahaya dan faktor-faktor lain dalam sistem pencahayaan rumah tangga.
6. Membandingkan efisiensi energi dan biaya pada sistem pencahayaan rumah tangga dengan teknologi kontrol otomatis dan sistem manual.
7. Menunjukkan implikasi dan rekomendasi dari hasil penelitian bagi rumah tangga dan lingkungan.

Contoh Tujuan Penelitian Skripsi Teknik Biomedis:

Berikut adalah beberapa contoh tujuan penelitian untuk penelitian dengan tema "Bagaimana memperbaiki akurasi deteksi kanker payudara dengan menggunakan teknologi pemindaian ultrasonografi berbasis AI":

1. Mengidentifikasi faktor-faktor yang mempengaruhi akurasi deteksi kanker payudara dengan menggunakan teknologi pemindaian ultrasonografi berbasis AI.
2. Menilai efektivitas teknologi pemindaian ultrasonografi berbasis AI dalam meningkatkan akurasi deteksi kanker payudara.
3. Menentukan metode pemindaian ultrasonografi berbasis AI yang paling efektif dalam meningkatkan akurasi deteksi kanker payudara.
4. Menilai keamanan dan tolerabilitas teknologi pemindaian ultrasonografi berbasis AI dalam deteksi kanker payudara.
5. Membandingkan akurasi deteksi kanker payudara dengan teknologi pemindaian ultrasonografi berbasis AI dengan metode deteksi lainnya.
6. Menyediakan bukti ilmiah untuk menunjukkan bahwa teknologi pemindaian ultrasonografi berbasis AI dapat digunakan sebagai metode deteksi kanker payudara yang lebih efektif dan akurat.
7. Meningkatkan akurasi deteksi kanker payudara dengan menggunakan teknologi pemindaian ultrasonografi berbasis AI.

Contoh Tujuan Penelitian Skripsi Teknologi Informasi:
Berikut adalah beberapa contoh tujuan penelitian untuk penelitian dengan tema "Bagaimana meningkatkan efisiensi dan keamanan sistem penyimpanan data cloud melalui implementasi teknologi enkripsi kuantum?":

1. Menilai efektivitas implementasi teknologi enkripsi kuantum dalam meningkatkan keamanan data pada sistem penyimpanan cloud.
2. Menentukan metode enkripsi kuantum yang paling efektif dalam meningkatkan keamanan data pada sistem penyimpanan cloud.
3. Membandingkan efisiensi enkripsi kuantum dengan metode enkripsi lainnya dalam meningkatkan keamanan data pada sistem penyimpanan cloud.
4. Menilai keamanan dan stabilitas sistem penyimpanan data cloud setelah implementasi teknologi enkripsi kuantum.
5. Menyediakan bukti ilmiah untuk menunjukkan bahwa implementasi teknologi enkripsi kuantum dapat meningkatkan efisiensi dan keamanan sistem penyimpanan data cloud.
6. Mengidentifikasi potensi masalah dan hambatan dalam implementasi teknologi enkripsi kuantum pada sistem penyimpanan data cloud.

1.4 Batasan Penelitian

Batasan penelitian teknik adalah pembatasan atau definisi yang menentukan lingkup dan ruang lingkup suatu penelitian dalam bidang teknik. Ini membantu membatasi masalah dan isu yang akan diteliti, menentukan metode dan teknik penelitian, membatasi populasi dan sampel, dan membatasi variabel dan hipotesis yang akan diuji. Batasan penelitian juga mencakup keterbatasan dan hambatan yang mungkin timbul selama proses penelitian. Tujuan utama dari batasan penelitian teknik adalah membuat penelitian terfokus dan terarah, serta memastikan hasil yang diperoleh dapat digunakan untuk menjawab pertanyaan dan masalah teknik secara efektif dan akurat.

Batasan penelitian dapat ditulis dengan format *list*. Contohnya adalah sebagai berikut:

1. Batasan 1.
2. Batasan 2.

1.5 Manfaat Penelitian

Manfaat penelitian didefinisikan sebagai manfaat yang diperoleh apabila skripsi telah selesai dilakukan. Manfaat skripsi pada umumnya berbentuk daftar. Manfaat

penelitian dapat berupa manfaat bagi dunia akademik dan atau masyarakat.

1. Enum 1
   - Coba 1
   - Coba 2
2. Enum 2

1.6 Sistematika Penulisan

Sistematika penulisan berisi pembahasan apa yang akan ditulis di setiap bab. Sistematika pada umumnya berupa paragraf yang setiap paragraf mencerminkan bahasan setiap Bab. Contoh:

Bab I membahas tentang pendahuluan yang berisi latar belakang, perumusan masalah dan tujuan penelitian.

Bab II berisi tentang metodologi penelitian yang terdiri dari desain penelitian, sumber data, Teknik pengumpulan data dan Teknik analisis data.

Dan seterusnya.

BAB II
TINJAUAN PUSTAKA DAN DASAR TEORI

2.1 Tinjauan Pustaka

Berisi tugas akhir-tugas akhir terdahulu yang terkait dengan judul skripsi yang dilakukan. Hal ini meliputi skripsi, tesis, atau publikasi terdahulu yang terkait dengan judul skripsi yang diusulkan. Lakukan pembahasan secara sistemastis dengan menjelaskan masalah apa yang dilakukan oleh tugas akhir terdahulu, kontribusi yang dilakukan, serta analisis penulis terkait dengan keunggulan dan keterbatasan tugas akhir.

Setelah membahas berbagai tugas akhir terdahulu, maka alangkah baiknya penulis melakukan rangkuman terutama terkait dengan peluang pengembangan atau tugas akhir yang akan dilakukan.

2.2 Dasar Teori

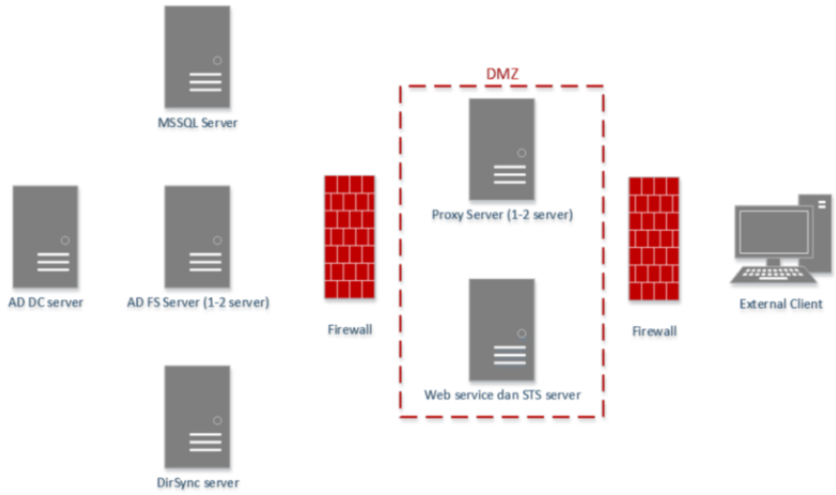
Berisi teori-teori yang menjadi dasar solusi atau produk hasil skripsi. Dasar teori pada umumnya diperoleh melalui buku referensi, publikasi tugas akhir, dan informasi web yang dapat dipertanggungjawabkan. Hindari penggunaan dasar teori melalui tautan wikipedia, surat kabar, atau portal berita.

2.2.1 Pengenalan Aplikasi Permainan

Proses pembuatan *game* dimulai dari pembuatan *game design document* dimana dokumen ini akan menjadi landasan pengembangan game tersebut serta menginformasikan gambaran keseluruhan game yang akan dibuat [**?**]. *Catatan: apapun yang diambil dari tulisan orang lain harus disitasi seperti dicontohkan [?].*

Game design document adalah sebuah bagian penting dalam pembuatan game baik itu elemen-elemen penyusunnya maupun proses pengembangannya. Game design yang telah dibuat, dijabarkan satu persatu mengenai tahapan dalam pembuatan game dan hasilnya disatukan dalam bentuk dokumentasi *game design document* yang digunakan oleh *developer* sebagai buku petunjuk bagaimana membuat *game* [**?**].

Dalam buku *Game Design Essentials* disebutkan *game design document* merupakan metode yang menghubungkan elemen-elemen penyusun *game*, baik itu *art, sound, program, gameplay* sehingga semuanya terdokumentasi menjadi satu dan menjadi acuan bagi para *developer* dalam membuat *game* [**?**].



Gambar 2.1. Contoh gambar [**?**]

2.2.2 Dasar Teori Lainnya

2.3 Analisis Perbandingan Metode

Di dalam tinjauan pustaka hasil akhirnya adalah analisis secara kualitatif atau pun secara kuantitatif kelebihan dan kekurangan metode jika dikaitkan dengan masalah, batasan-batasan masalah dan solusi yang dinginkan. Analisis kuantitatif tidak wajib teapi mempunyai nilai tambah di dalam tugas akhir saudara. Bagian ini menjelaskan kenapa metode tersebut dipilih dan uraikan dengan lebih jelas metode pelaksanaan tugas akhir yang ingin Anda lakukan.

2.4 Pertanyaan Tugas Akhir (Jika Perlu)

Pertanyaan tugas akhir bersifat opsional dan dapat ditambahkan untuk menekankan hal-hal yang hendak diketahui dari tugas akhir berdasar pada tujuan tugas akhir. Pertanyaan tugas akhir dikenal dengan RQ (*Research Question*) dan harus memiliki keterkaitan dengan RO (*Research Objective*). Satu RO dapat memiliki satu atau lebih dari satu RQ.

BAB III
METODE PENELITIAN

Bab ini menjelaskan mengenai metodologi yang digunakan dalam tahap implementasi dan verifikasi *System-on-Chip* (SoC) berbasis *soft-core* mikroprosesor mriscv pada *board* FPGA Digilent Basys3 berbasis chip Xilinx Artix-7, dengan memanfaatkan *open-source synthesis toolchain*. Pembahasan meliputi alat dan bahan yang digunakan, metode yang diterapkan dalam penelitian, alur pelaksanaan tugas akhir, struktur hierarki RTL yang menyusun *soft-core* mriscv, dan metode analisis hasil akhir.

3.1 Alat dan Bahan Tugas Akhir

Penelitian tugas akhir ini memanfaatkan kombinasi *hardware* fisik dan *software* untuk melakukan implementasi, verifikasi, serta evaluasi. Rincian mengenai alat dan bahan yang digunakan dalam penelitian ini ditunjukkan sebagai berikut:

3.1.1 Alat Tugas Akhir (*Hardware*)

Hardware utama yang digunakan sebagai bahan eksperimen meliputi:

1. *Board* FPGA Digilent Basys3 dengan chip Xilinx Artix-7 XC7A35T-1CPG236C dengan spesifikasi 33.280 *logic cell*, 1.800 Kbits *Block RAM* internal, dan 50 slot I/O. *Board* ini dilengkapi dengan 16 *switch*, 5 *push button*, *seven-segment display* 4 digit, 16 LED, dan konektor USB untuk pemrograman dan komunikasi serial. Konektor USB tersebut terhubung ke *bridge* USB FTDI FT2232HQ pada *board*. Salah satu *channel*-nya muncul sebagai *virtual COM port* pada komputer dan digunakan untuk komunikasi UART antara *soft-core* dan komputer.
2. Total Phase Cheetah SPI Adapter sebagai SPI *master* untuk mentransfer *firmware binary* dari PC ke memori mriscv saat *runtime*. *Adapter* ini mendukung *data transfer rate* hingga 40 MHz dan menyediakan *USB interface* untuk komunikasi dengan komputer.
3. Komputer dengan *operating system* Linux Ubuntu 24.04 LTS atau lebih baru.

3.1.2 Bahan Tugas Akhir

Bahan yang digunakan dalam penelitian tugas akhir ini sebagai berikut:

1. Yosys Open SYnthesis Suite yang digunakan sebagai *synthesis engine* untuk mentransformasikan deskripsi *Register Transfer Level* (RTL) Verilog menjadi *gate-level netlist* yang tersusun atas *primitive* FPGA Xilinx.

2. nextpnr-xilinx sebagai *place-and-route tool* (PnR) untuk memetakan *gate-level netlist* ke *physical resource* pada chip FPGA Xilinx Artix-7.
3. openXC7 Toolchain sebagai *toolchain* untuk mengonversi hasil *place-and-route* menjadi *bitstream file* yang dapat diprogram ke FPGA.
4. mriscv sebagai *core* mikroprosesor RISC-V yang digunakan sebagai *processing core* utama dalam desain SoC.
5. openFPGALoader sebagai *software utility* untuk *flashing bitstream* dari PC ke *configuration memory* SRAM FPGA melalui *JTAG interface*.
6. riscv64-unknown-elf-gcc Toolchain sebagai *toolchain cross-compiler* yang mengompilasi *source code* C/C++ menjadi kode *binary* yang dapat dijalankan pada target *instruction set architecture* RISC-V RV32I.
7. Icarus Verilog (`iverilog`) sebagai simulator Verilog untuk menjalankan *testbench* fungsional, baik untuk satu modul maupun untuk SoC lengkap yang menjalankan *firmware* hasil kompilasi, sebelum desain disintesis.
8. *Serial terminal* pada komputer untuk mengirim dan menerima data UART melalui *virtual COM port* dengan *baud rate* 115200.

3.2 Metode yang Digunakan

Perancangan dan penerapan pada penelitian tugas akhir ini dilaksanakan secara langsung, dengan berpusat pada uji coba menggunakan *development board* FPGA sebagai objek eksperimen untuk menguji dan validasi desain *soft-core* mriscv yang telah dimodifikasi untuk berjalan pada *board* FPGA Digilent Basys3. Validitas hasil diperoleh melalui proses sintesis aktual dan pengujian langsung pada *hardware* fisik.

Tahap analisis pada penelitian ini dilakukan melalui dua pendekatan, yaitu analisis validitas fungsional dan analisis kuantitatif terhadap hasil sintesis. Analisis validitas fungsional dilakukan dengan membandingkan perilaku aktual *soft-core* pada *hardware* terhadap perilaku yang diharapkan berdasarkan spesifikasi program yang diunggah, guna memastikan kebenaran implementasi. Analisis kuantitatif dilakukan melalui *benchmarking* terhadap hasil sintesis, mencakup *timing analysis* untuk memperoleh parameter *operating frequency* dan *maximum frequency* (*Fmax*) serta evaluasi *resource utilization* pada FPGA.

3.3 Alur Tugas Akhir

Penulisan tugas akhir ini dilakukan melalui beberapa tahapan yang berlangsung secara berurutan. Kerangka metodologis ini disusun agar setiap tahapan menjadi prasyarat bagi tahapan berikutnya, sehingga proses penelitian dapat ditelusuri secara sistematis.

Penjelasan rinci untuk setiap tahapan diuraikan sebagai berikut.

1. **Studi Literatur dan Analisis RTL *Repository* mriscv**

   Tahap ini diawali dengan studi literatur mengenai arsitektur *soft-core* prosesor RISC-V mriscv, mencakup pemahaman terhadap *instruction set* RV32I [1], hierarki modul penyusun *core* (antara lain *control unit* berbasis *multi-cycle finite state machine*, *arithmetic logic unit*, *register file*, dan *memory interface*), serta arsitektur bus AXI4-Lite yang menghubungkan *core* dengan modul periferalnya. Berdasarkan pemahaman tersebut, dilakukan analisis awal terhadap kode RTL *repository* mriscv untuk mengidentifikasi bagian desain yang berpotensi tidak kompatibel dengan *routing fabric* FPGA target, seperti konstruksi bus *tri-state*, struktur *latch*, dan hal lainnya. Hasil analisis ini menjadi dasar penentuan bagian RTL yang perlu dimodifikasi pada tahap berikutnya.

2. **Modifikasi dan Implementasi *Hardware***

   Berdasarkan hasil analisis yang diperoleh, dilakukan modifikasi pada kode RTL yang memerlukan penyesuaian untuk mendukung kompatibilitas antara desain *soft-core* mriscv dengan *routing fabric* FPGA target. Modifikasi yang dilakukan mencakup penyesuaian modul memori SRAM menjadi bentuk *Block RAM* (BRAM) dengan blok inisialisasi nilai awal, penyesuaian struktur modul *interconnect* untuk memastikan *timing* yang andal dan menghindari *timing violation* pada hasil sintesis, serta penggantian struktur bus *tri-state* pada *write-back path* register di dalam `mriscvcore` dan ALU menjadi struktur *multiplexer* agar dapat disintesis dengan benar pada *routing fabric* FPGA target. Setelah kode RTL dimodifikasi, selanjutnya dilakukan proses sintesis menggunakan *open-source toolchain*, yaitu Yosys untuk menghasilkan *gate-level netlist* dan nextpnr-xilinx untuk proses *place-and-route* pada *board* FPGA Digilent Basys3. Hasil *place-and-route* kemudian dikonversi menjadi *bitstream file* dan diprogramkan ke FPGA menggunakan openFPGALoader.

   Setelah *soft-core* berjalan stabil, rancangan diperluas dengan tiga modul periferal baru yang ditulis dari awal karena tidak tersedia pada *repository* mriscv, yaitu UART *transmitter* (`uart_tx.v`), UART *receiver* (`uart_rx.v`), dan I2C *master controller* (`i2c_master.v`). Ketiga modul dirancang sebagai *slave* AXI4-Lite agar dapat dihubungkan ke *interconnect bus* tanpa mengubah *core*. Setiap modul diverifikasi terlebih dahulu dengan *testbench* pada simulator `iverilog` sebelum disintesis dan diuji pada *hardware*.

3. **Pengembangan *Firmware***

   Tahap ini mencakup pengembangan *bare-metal firmware* yang berjalan pada *soft-core* mriscv tanpa *operating system*. Pengembangan diawali dengan

penyusunan *startup code* yang menyiapkan kondisi awal prosesor sebelum program utama dijalankan, dilanjutkan dengan penyusunan *linker script* untuk mengatur *memory map* program sesuai dengan kapasitas BRAM yang tersedia. Selanjutnya dibuat *header* C yang berfungsi sebagai *hardware abstraction layer* yang memuat fungsi dasar untuk mengakses register periferal, seperti GPIO, UART, dan I2C *controller*. Seluruh kode program dikompilasi menggunakan *toolchain* riscv64-unknown-elf-gcc yang sesuai dengan arsitektur RV32I.

4. **Integrasi dan Pengujian Sistem**

   Tahap ini merupakan tahap integrasi antara *hardware* dan *software*, di mana *firmware* yang telah dikompilasi dimuat ke memori *soft-core* mriscv dan diuji secara fungsional. Pada tahap pengembangan awal, *firmware loading* dilakukan dengan menyisipkan kode *binary* hasil kompilasi langsung ke dalam parameter inisialisasi (`INIT_FILE`) modul BRAM pada tahap sintesis RTL, sehingga untuk mengubah kode yang berjalan pada *soft-core* mriscv diperlukan proses sintesis ulang dari awal secara menyeluruh.

   Setelah dipastikan bahwa *hardware* dan *software* telah berjalan dengan baik, selanjutnya ditambahkan mekanisme *runtime loader* dengan *Serial Peripheral Interface* (SPI) menggunakan Total Phase Cheetah SPI Adapter. Mekanisme ini memungkinkan *firmware* yang telah dikompilasi dimuat ke memori *soft-core* mriscv tanpa perlu melakukan sintesis ulang *hardware* setiap kali ingin dilakukan perubahan program.

   Selanjutnya, dilakukan pengujian fungsional menggunakan modul *rotary encoder* KY-040 sebagai *input device*, dan OLED SSD1306 sebagai *output device* yang berkomunikasi dengan *soft-core* mriscv melalui protokol I2C. Komunikasi I2C dengan layar OLED diuji dengan dua cara, yaitu secara *bit-bang* melalui GPIO dan melalui *hardware* I2C *controller*. UART *interface* diuji dengan mengirim teks dari *soft-core* ke *serial terminal* pada komputer serta melakukan *echo* terhadap karakter yang diterima dari terminal. Pengujian dilakukan untuk memastikan bahwa seluruh skenario pengujian berjalan sesuai dengan yang diharapkan, mencakup *firmware loading*, eksekusi instruksi RV32I, pembacaan arah putaran dari *rotary encoder*, pengendalian tampilan pada layar OLED, serta pertukaran data melalui UART.

5. **Pengumpulan Data dan Analisis**

   Tahap terakhir adalah pengumpulan data dan analisis hasil implementasi. Tahap ini diawali dengan validasi fungsional untuk memastikan seluruh skenario pengujian pada tahap sebelumnya berjalan sesuai dengan yang diharapkan. Selanjutnya dilakukan *benchmarking* terhadap hasil sintesis, yang mencakup evaluasi *resource utilization* FPGA seperti *Look-Up Table* (LUT), *Flip-Flop* (FF), dan *Block RAM*

(BRAM), serta *timing analysis* untuk memperoleh nilai *maximum frequency* (*Fmax*) yang dapat dicapai oleh desain. Data yang diperoleh dari kedua proses tersebut selanjutnya dianalisis dan dibahas secara mendalam pada bab selanjutnya.

3.4 Struktur Hierarki *Soft-core* mriscv

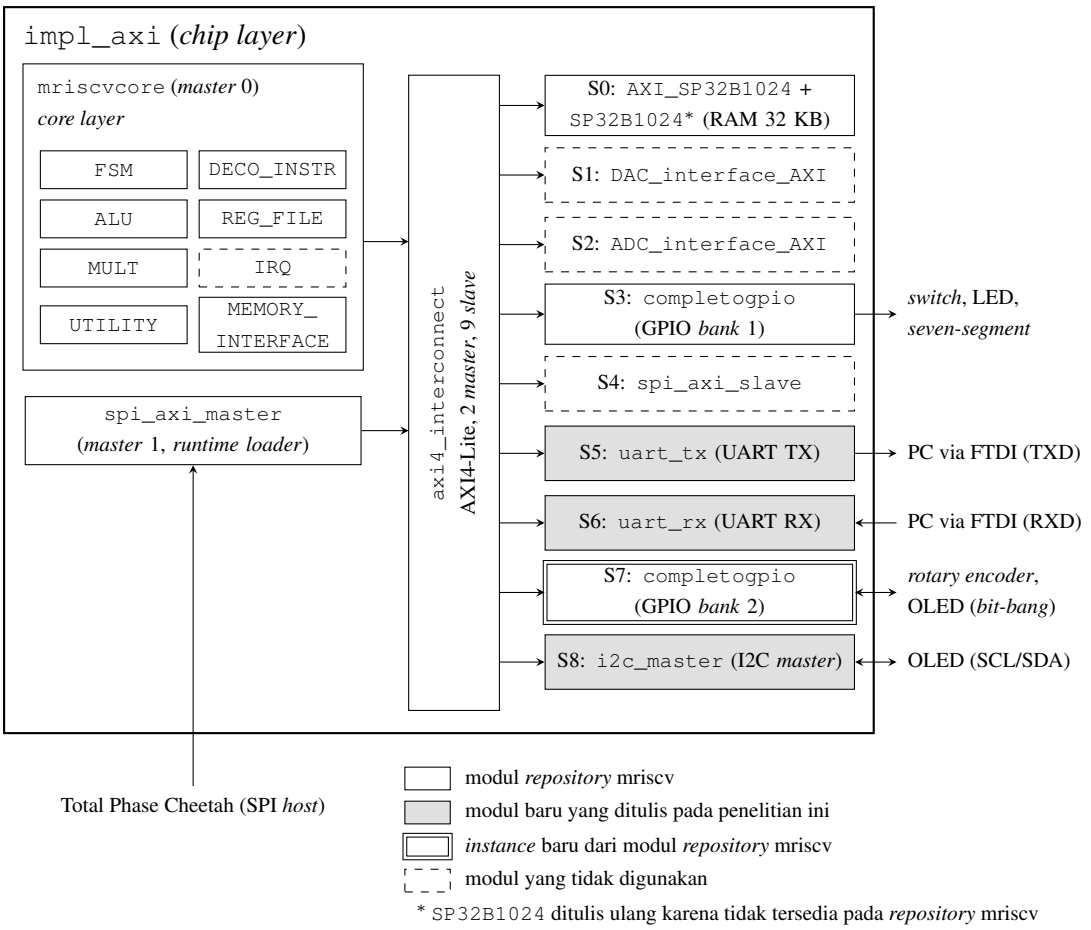
Subbab ini menguraikan struktur hierarki rancangan RTL (*Register Transfer Level*) yang menyusun *soft-core* mriscv, mulai dari *central processing unit* (CPU `mriscvcore`) beserta komponen-komponen penyusunnya seperti *Arithmetic Logic Unit* (ALU) dan *register file*, *interconnect bus*, hingga modul periferal yang digunakan pada penelitian ini.

3.4.1 Hierarki Modul RTL Verilog

Secara keseluruhan, modul RTL pada *repository* mriscv tersusun atas tiga *layer* hierarki, yaitu *chip layer*, *core layer*, dan *layer* modul fungsional penyusun *core*. Susunan hierarki tersebut ditunjukkan sebagai berikut.

```
impl_axi.v
├── mriscvcore.v
│   ├── FSM.v
│   ├── DECO_INSTR.v
│   ├── ALU.v
│   ├── REG_FILE.v
│   ├── MEMORY_INTERFACE.v
│   ├── MULT.v
│   ├── IRQ.v
│   └── UTILITY.v
├── spi_axi_master.v
├── axi4_interconnect.v
├── AXI_SP32B1024.v, SP32B1024.v
├── DAC_interface_AXI.v
├── ADC_interface_AXI.v
├── completogpio.v (GPIO bank 1)
├── spi_axi_slave.v
├── uart_tx.v
├── uart_rx.v
├── completogpio.v (GPIO bank 2)
└── i2c_master.v
```

Gambar 3.2 menunjukkan hierarki yang sama dalam bentuk diagram blok, beserta hubungan setiap modul melalui bus AXI4-Lite. Notasi S0 sampai S8 pada gambar menyatakan *slave* ke-0 sampai ke-8, dan nama pada gambar adalah nama modul Verilog tanpa ekstensi `.v`.



Gambar 3.2. Diagram Hierarki Modul RTL dan Hubungannya melalui Bus AXI4-Lite

Modul `impl_axi.v` merupakan *chip layer* yang menjadi *wrapper* instansiasi bagi seluruh modul utama pada struktur hierarki di atas, mulai dari *core* mriscv, SPI *interface* untuk *runtime loader*, *interconnect bus*, hingga modul-modul periferal, yang seluruhnya saling terhubung melalui bus AXI4-Lite.

Modul `mriscvcore.v` merupakan *central processing unit* (CPU) sekaligus sebagai *master* bus ke-0. Modul ini tersusun atas beberapa modul fungsional, antara lain `FSM.v` yang berfungsi sebagai *control unit* berbasis *multi-cycle finite state machine*, `DECO_INSTR.v` sebagai *decoder* instruksi RV32I, `ALU.v` sebagai *arithmetic logic unit*, `REG_FILE.v` sebagai *register file*, `MEMORY_INTERFACE.v` sebagai *memory/bus interface* untuk instruksi *load*/*store*, `MULT.v` sebagai *multiplier*, `IRQ.v` sebagai unit *timer-based interrupt*, dan `UTILITY.v` sebagai unit instruksi *utility*/sistem.

Selain `mriscvcore.v`, terdapat beberapa modul penyusun lain pada *chip layer*. Modul `spi_axi_master.v` berfungsi sebagai SPI *interface* sekaligus *master* bus ke-1. Modul ini berperan sebagai *runtime loader* yang menghubungkan *soft-core* dengan Total Phase Cheetah SPI Adapter (sebagai *host* eksternal) untuk memuat

program ke dalam modul memori. Selanjutnya, modul `axi4_interconnect.v` bertindak sebagai *interconnect bus* AXI4-Lite yang menghubungkan kedua *master* tersebut dengan sembilan modul *slave*. Lima *slave* pertama merupakan bagian dari desain asli mriscv, meliputi `AXI_SP32B1024.v` dan `SP32B1024.v` sebagai modul memori (*slave* ke-0), `DAC_interface_AXI.v` sebagai DAC *interface* (*slave* ke-1, tidak digunakan pada penelitian ini), `ADC_interface_AXI.v` sebagai ADC *interface* (*slave* ke-2, tidak digunakan pada penelitian ini), `completogpio.v` sebagai modul GPIO (*slave* ke-3), dan `spi_axi_slave.v` sebagai SPI *master interface* yang dikendalikan *core* untuk mengirim data ke perangkat SPI eksternal (*slave* ke-4, tidak digunakan pada penelitian ini). Seluruh modul untuk kelima *slave* tersebut berasal dari *repository* mriscv, kecuali `SP32B1024.v` yang tidak tersedia pada *repository* sehingga ditulis ulang pada penelitian ini (Subbab 4.4.1). Empat *slave* berikutnya ditambahkan pada penelitian ini. Tiga di antaranya merupakan modul baru yang ditulis pada penelitian ini, yaitu `uart_tx.v` sebagai UART *transmitter* (*slave* ke-5), `uart_rx.v` sebagai UART *receiver* (*slave* ke-6), dan `i2c_master.v` sebagai I2C *master controller* (*slave* ke-8). *Slave* ke-7 merupakan instansiasi kedua `completogpio.v` dari *repository* mriscv yang dipakai sebagai GPIO *bank* 2.

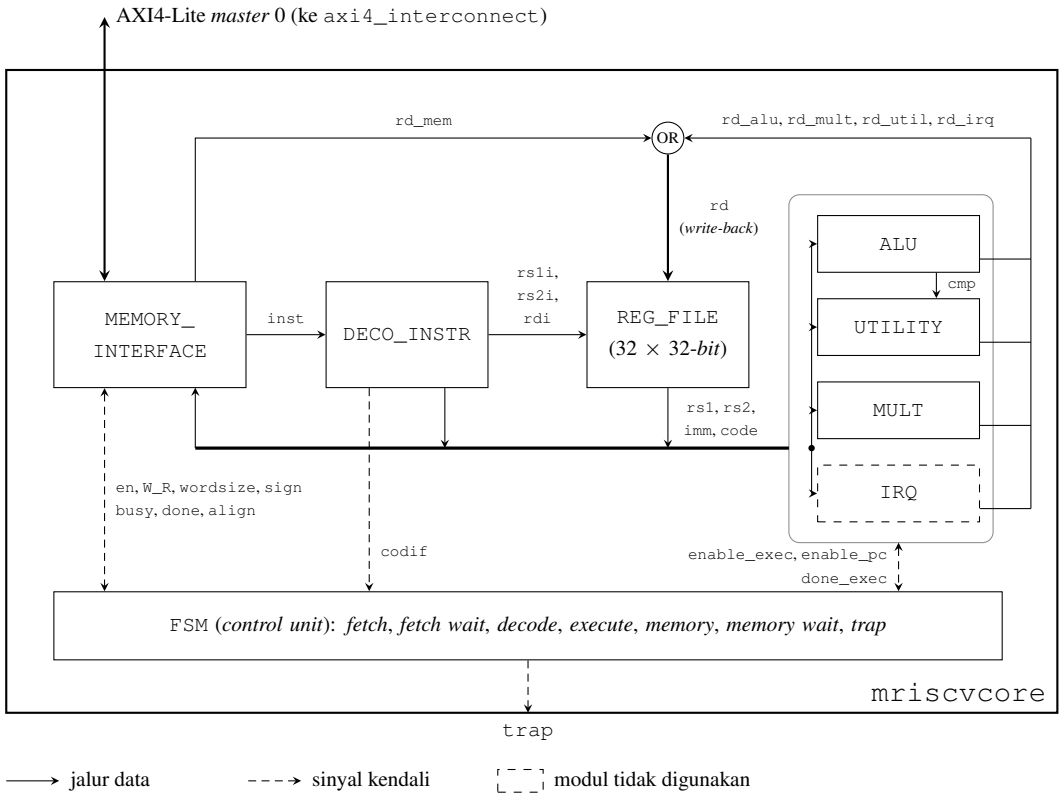
3.4.2 Struktur Internal `mriscvcore`

Processing unit utama pada mriscv, yaitu `mriscvcore.v`, bekerja berdasarkan arsitektur *multi-cycle* yang dikendalikan oleh *control unit* `FSM.v`. *Control unit* ini tersusun atas tujuh *state* yang dieksekusi berurutan untuk setiap instruksi: *fetch*, *decode*, *execute*, dan *memory access*. Ditambahkan pula *wait state* pada tahap *fetch* dan *memory access* untuk mengakomodasi latensi BRAM yang bersifat *synchronous*, serta satu *state trap* yang menghentikan eksekusi secara permanen ketika ditemukan instruksi ilegal atau *bus error*. Karena bersifat *multi-cycle* dan tidak menerapkan *pipeline*, setiap instruksi membutuhkan lebih dari satu *clock cycle* untuk selesai dieksekusi (*Cycles Per Instruction*/CPI > 1). Oleh karena itu, *hazard control unit* antarinstruksi tidak diperlukan pada arsitektur ini [2, 3].

`DECO_INSTR.v` menerjemahkan instruksi yang telah diambil pada tahap *fetch* untuk menentukan jenis operasi dan operand yang digunakan. Operand tersebut diambil dari `REG_FILE.v`, *register file* yang menyimpan 32 register 32-*bit* sesuai spesifikasi arsitektur RV32I. `ALU.v` menangani komputasi aritmetika dan logika, sementara akses ke memori maupun periferal pada bus berlangsung melalui `MEMORY_INTERFACE.v`. `MULT.v` menangani instruksi perkalian `mul`, `mulh`, `mulhsu`, dan `mulhu` dari ekstensi Zmmul menggunakan *Booth multiplier* iteratif. Seperti `ALU.v`, `MULT.v` diaktifkan oleh sinyal `enable_exec` dari `FSM.v` pada tahap *execute*. Bedanya, perkalian membutuhkan beberapa *clock cycle*, sehingga logika pada `mriscvcore.v` baru menyatakan tahap *execute* selesai (`done_exec`) setelah `MULT.v` mengeluarkan sinyal

done. Dengan demikian, *core* mriscv mendukung RV32I ditambah ekstensi Zmmul, yaitu perkalian tanpa pembagian. *Firmware* bawaan pada penelitian ini dikompilasi dengan `-march=rv32i`, sehingga operator perkalian pada kode C dijalankan oleh rutin *software* `__mulsi3`. Instruksi perkalian *hardware* baru dipakai bila *firmware* dikompilasi dengan `-march=rv32i_zmmul`. *Repository* mriscv juga menyertakan `IRQ.v` untuk penanganan *interrupt timer*, tetapi *output interrupt*-nya tidak disambungkan pada `impl_axi.v`, sehingga `IRQ.v` tidak dipakai pada penelitian ini. *Output* dari kelima modul tersebut, yaitu `ALU.v`, `MEMORY_INTERFACE.v`, `MULT.v`, `IRQ.v`, dan `UTILITY.v`, digabungkan melalui satu *shared bus* menuju `REG_FILE.v` untuk proses *write-back*.

Gambar 3.3 menunjukkan hubungan antarmodul fungsional di dalam `mriscvcore.v`. Garis tebal horizontal menyatakan bus operand (`rs1`, `rs2`, `imm`, dan `code`) yang dibagikan ke seluruh unit eksekusi dan `MEMORY_INTERFACE.v`, sedangkan simpul OR menyatakan penggabungan *output* unit-unit tersebut menjadi sinyal `rd` yang ditulis kembali ke `REG_FILE.v`. Nama sinyal pada gambar mengikuti nama *wire* pada `mriscvcore.v`.



Gambar 3.3. Diagram Blok Struktur Internal `mriscvcore`

3.4.3 Bus AXI4-Lite dan Modul Periferal

Seluruh modul pada *chip layer* saling terhubung melalui `axi4_interconnect.v`, *interconnect bus* bertipe AXI4-Lite yang menghubungkan

dua *master* dan sembilan *slave*. `mriscvcore.v` bertindak sebagai *master* ke-0 dan `spi_axi_master.v` sebagai *master* ke-1, sedangkan kesembilan *slave* beserta *address map*-nya ditunjukkan pada Tabel 3.1. *Address* pada tabel merupakan *word address* (32-*bit*). *Byte address* yang dipakai *firmware* sama dengan empat kali *word address*.

Slave	**Periferal**	***Word Address***	**Keterangan**
0	Memori (RAM)	`0x0000–0x1FFF`	Digunakan untuk penyimpanan program dan data
1	DAC	`0x4000–0x4001`	Tidak digunakan pada penelitian ini
2	ADC	`0x4008–0x4009`	Tidak digunakan, *output* ditetapkan nol
3	GPIO *bank* 1	`0x4010–0x401F`	*Switch*, LED, dan *seven-segment*
4	SPI *interface*	`0x04000000`	Tidak digunakan pada penelitian ini
5	UART TX	`0x4020`	UART *transmitter* (ditambahkan)
6	UART RX	`0x4021`	UART *receiver* (ditambahkan)
7	GPIO *bank* 2	`0x4030–0x403F`	*Rotary encoder* dan *line* I2C OLED secara *bit-bang* (ditambahkan)
8	I2C *master*	`0x4040–0x4043`	*Hardware* I2C *controller* (ditambahkan)

Tabel 3.1. *Address Map* Bus AXI4-Lite

Slave ke-0 merupakan modul memori yang tersusun atas `AXI_SP32B1024.v` sebagai *bus interface* dan `SP32B1024.v` sebagai modul memori yang dipetakan ke *Block RAM* (BRAM) FPGA untuk menyimpan program *firmware* beserta data selama eksekusi. *Slave* ke-1 dan ke-2 masing-masing adalah DAC *interface* dan ADC *interface* yang tersedia pada *repository* mriscv, tetapi tidak dimanfaatkan pada penelitian ini karena tidak ada kebutuhan *analog signal processing*. *Slave* ke-4, `spi_axi_slave.v`, merupakan SPI *master interface* yang memungkinkan *core* mengirim data ke perangkat SPI eksternal. *Output*-nya tidak dihubungkan ke pin *board*, sehingga modul ini juga tidak digunakan. Modul ini berbeda dari *SPI runtime loader*: `spi_axi_master.v` berkedudukan sebagai *master* ke-1 yang menulis *firmware* langsung ke memori (*slave* ke-0) tanpa melalui `mriscvcore.v`.

Modul `completogpio.v` merupakan modul periferal *input-output* digital (GPIO) yang diinstansiasi dua kali. Instansiasi pertama (*slave* ke-3, GPIO *bank* 1) terhubung ke *switch*, LED, dan *seven-segment* pada *board*. Instansiasi kedua (*slave*

ke-7, GPIO *bank* 2) ditambahkan agar *external device* tidak berbagi indeks pin dengan *bank* 1. Dua pin *input bank* 2 membaca sinyal CLK dan DT dari *rotary encoder* KY-040, yang didekode dengan metode *quadrature decoding* berbasis *software*, sedangkan dua pin *output*-nya men-*drive line* SCL dan SDA layar OLED SSD1306 untuk komunikasi I2C secara *bit-bang*.

Komunikasi serial dengan komputer ditangani oleh `uart_tx.v` (*slave* ke-5) dan `uart_rx.v` (*slave* ke-6), yang terhubung ke *channel* serial USB pada *board*. Modul `i2c_master.v` (*slave* ke-8) menyediakan *hardware* I2C *controller* yang memakai *line* SCL dan SDA yang sama dengan GPIO *bank* 2, sehingga layar OLED dapat dikendalikan secara *bit-bang* maupun melalui *hardware controller* tanpa mengubah *circuit*. Rancangan ketiga modul tersebut diuraikan pada Subbab 3.4.4.

3.4.4 Rancangan Modul Periferal UART dan I2C

Modul `uart_tx.v`, `uart_rx.v`, dan `i2c_master.v` ditulis dari awal sebagai *slave* AXI4-Lite dengan pola *interface* yang sama: *write channel* dan *read channel* masing-masing dikendalikan oleh FSM kecil yang menyelesaikan setiap transaksi bus dalam beberapa *clock cycle*. Pola ini disebut *non-blocking*. Pengiriman satu *byte* UART dengan *baud rate* 115200 memakan sekitar 4.340 *clock cycle* pada *clock core*, tetapi transaksi AXI ke modul tetap selesai dalam beberapa *cycle*, sehingga periferal yang lambat tidak menahan *interconnect bus* yang juga dipakai *SPI runtime loader*. Proses menunggu periferal siap dipindahkan ke *firmware*, yang melakukan *polling* terhadap *bit* status *busy* sebelum menulis data berikutnya. Data yang ditulis ketika modul masih *busy* akan diabaikan. Ketiga modul menerima parameter `CLK_HZ` yang harus sama dengan frekuensi *clock core* (50 MHz), karena *baud rate* UART dan frekuensi SCL I2C diturunkan dari parameter tersebut.

Modul `uart_tx.v` mengirim data dengan format 8N1 (8 *bit* data, *no parity*, 1 *stop bit*) secara LSB-*first* dengan *baud rate* tetap 115200. *Bit period* sebesar $\lfloor 50\,000\,000/115\,200 \rfloor = 434$ *clock cycle*, sehingga *baud rate* aktual 115.207 dengan *error* sekitar 0,0064%. *Transmitter* dikendalikan FSM dengan empat *state* (IDLE, START, DATA, STOP). *Output* TXD dihubungkan ke pin A18 *board*.

Modul `uart_rx.v` menerima data dengan format yang sama melalui pin B18. Sinyal RXD yang bersifat *asynchronous* terhadap *clock core* dilewatkan *two-flip-flop synchronizer*. Setelah *falling edge start bit* terdeteksi, *receiver* menunggu setengah *bit period* lalu memeriksa ulang RXD. Bila RXD sudah kembali *high*, pulsa tersebut dianggap *glitch* dan diabaikan. Setiap *bit* data kemudian di-*sample* pada *mid-bit*, dan *stop bit* yang bernilai nol dicatat sebagai *framing error*. *Byte* yang diterima disimpan pada *holding register* satu *byte* bersama tiga *flag*: *valid*, *overrun* (ada *byte* yang tertimpa sebelum sempat dibaca), dan *framing error*. Pembacaan register sekaligus melakukan

clear terhadap ketiga *flag* tersebut.

Modul `i2c_master.v` merupakan I2C *master controller byte-level*. Satu penulisan ke register CMD menjalankan satu operasi dengan urutan tetap: [START atau *repeated* START] → [kirim satu *byte* atau terima satu *byte*] → [STOP], dengan setiap bagian dipilih melalui *bit* pada CMD. Setiap *bit* I2C dibagi menjadi empat *phase*, dan panjang satu *phase* ditentukan register DIV dalam satuan *clock cycle*. Nilai *default*-nya 31, sehingga frekuensi SCL sekitar 400 kHz (*Fast-mode*). *Controller* ini membaca ACK/NACK dari *slave*, mendukung *clock stretching* (*controller* menunggu SCL benar-benar *high* sebelum melanjutkan), dan mendeteksi *arbitration lost*.

Line SCL dan SDA dibangun sebagai *open-drain* pada *top module* dengan logika *wired-AND*: *line* bernilai *low* apabila salah satu dari GPIO *bank* 2 atau `i2c_master.v` men-*drive low*, dan bernilai *high* melalui *pull-up* apabila keduanya melakukan *release*. Dengan cara ini *firmware* I2C *bit-bang* yang sudah ada tetap dapat dipakai tanpa *multiplexer* tambahan. Konsekuensinya, karena GPIO *bank* 2 bernilai nol setelah *reset*, *firmware* yang memakai *hardware controller* harus menulis logika 1 ke kedua pin GPIO *bank* 2 terlebih dahulu. *Register map* ketiga modul ditunjukkan pada Tabel 3.2.

Modul	***Byte Address***	**Akses**	**Fungsi**
UART TX	`0x10080`	*Write*	`[7:0]`: *byte* yang dikirim
		Read	`[0]`: *busy*
UART RX	`0x10084`	*Read*	`[7:0]`: data, `[8]`: *valid*, `[9]`: *overrun*, `[10]`: *framing error*. *Flag* di-*clear* saat dibaca
I2C DATA	`0x10100`	*Write*/*Read*	*Write*: *byte* yang dikirim. *Read*: *byte* terakhir yang diterima
I2C CMD	`0x10104`	*Write*	`[0]` START, `[1]` STOP, `[2]` WRITE, `[3]` READ, `[4]` NACK setelah *byte* yang dibaca
I2C STATUS	`0x10108`	*Read*	`[0]`: *busy*, `[1]`: NACK dari *slave*, `[2]`: *arbitration lost*
I2C DIV	`0x1010C`	*Write*/*Read*	Jumlah *clock cycle* per seperempat periode SCL (minimum 4, *default* 31)

Tabel 3.2. *Register Map* Modul UART dan I2C

Verifikasi ketiga modul dilakukan dalam tiga *level*. *Level* pertama adalah *unit testbench* pada `iverilog` untuk setiap modul, mencakup kondisi normal dan kondisi *error* seperti *overrun*, *framing error*, *glitch*, NACK, *clock stretching*, dan *arbitration lost*. *Level* kedua adalah simulasi SoC lengkap, yaitu *testbench* yang memuat *firmware* hasil kompilasi melalui *frame* SPI lalu menjalankannya pada *core* mriscv. *Level* ketiga adalah pengujian pada *board* menggunakan *firmware* uji, dengan hasil diamati melalui *serial terminal*, LED, layar OLED, dan LED indikator *trap* (LD15).

3.5 Metode Analisis Hasil Akhir

Analisis terhadap hasil akhir penelitian ini dilakukan melalui dua pendekatan, yaitu analisis fungsional dan analisis kuantitatif terhadap hasil sintesis.

3.5.1 Analisis Fungsional

Analisis fungsional dilakukan dengan membandingkan perilaku aktual sistem terhadap perilaku yang diharapkan pada lima sasaran pengujian. Pada GPIO *interface* untuk LED, kebenaran fungsional diukur dari kesesuaian pola nyala LED yang dihasilkan *firmware* terhadap pola yang telah ditentukan pada program yang diunggah. Pada *seven-segment display*, kebenaran fungsional diukur dari kesesuaian digit yang ditampilkan terhadap nilai yang seharusnya ditampilkan berdasarkan *firmware* uji yang dijalankan.

Pada sasaran pengujian menggunakan *rotary encoder* KY-040, kebenaran fungsional diukur dari kesesuaian arah putaran (*CW*/*CCW*) yang terbaca oleh *firmware* melalui metode *quadrature decoding* terhadap arah putaran fisik yang sebenarnya dilakukan pada *encoder*. Pada layar OLED SSD1306, kebenaran fungsional diukur dari kesesuaian tampilan pada layar terhadap data yang dikirimkan oleh *firmware* melalui protokol I2C, baik secara *bit-bang* maupun melalui *hardware* I2C *controller*, mencakup keberhasilan inisialisasi layar maupun kesesuaian karakter atau grafik yang ditampilkan. Untuk *hardware* I2C *controller*, kriteria ditambah dengan status ACK yang dibaca langsung oleh *hardware* pada setiap *byte*: selama pengujian tidak boleh ada *byte* yang dijawab NACK maupun kejadian *arbitration lost*.

Pada UART *interface*, kebenaran fungsional *transmitter* diukur dari kesesuaian teks yang diterima *serial terminal* pada komputer terhadap *string* yang dikirim *firmware*, termasuk pada pengiriman terus-menerus tanpa jeda. Kebenaran fungsional *receiver* diukur dari kesesuaian karakter hasil *echo* oleh *firmware* terhadap karakter yang diketik pada terminal. Implementasi dinyatakan berhasil secara fungsional apabila *output* sistem pada kelima sasaran pengujian tersebut sesuai dengan hasil yang diharapkan. Apabila ditemukan ketidaksesuaian pada salah satu sasaran pengujian, dilakukan penelusuran lebih lanjut terhadap penyebabnya.

3.5.2 Analisis Kuantitatif

Analisis kuantitatif dilakukan terhadap dua parameter utama hasil sintesis, yaitu *maximum operating frequency* (*Fmax*) dan *resource utilization* FPGA. Nilai *Fmax* diperoleh dari laporan *timing analysis* yang dihasilkan oleh nextpnr-xilinx pada setiap proses sintesis. Nilai *Fmax* dicetak dua kali pada log *output* nextpnr-xilinx, sehingga nilai yang diambil sebagai data adalah laporan kedua yang bersifat final setelah tahap

routing selesai. Karena proses *place-and-route* pada nextpnr-xilinx bersifat stokastik terhadap nilai *seed* yang digunakan [4], analisis dilakukan melalui dua skema eksperimen. Skema pertama adalah pengujian *repeatability*, yaitu proses sintesis diulang beberapa kali pada konfigurasi *seed* yang sama, untuk menguji konsistensi hasil pada kondisi *place-and-route* yang deterministik. Skema kedua adalah pengujian sensitivitas *seed*, yaitu proses sintesis diulang menggunakan beberapa nilai *seed* eksplisit yang berbeda, untuk menguji variabilitas nilai *Fmax* yang dapat timbul akibat proses *placement* dan *routing* yang berbeda pada RTL yang sama. Nilai *Fmax* rata-rata dan standar deviasi dari kedua skema tersebut selanjutnya dibandingkan terhadap *target operating frequency* sistem, guna menentukan *safety margin* desain.

Resource utilization FPGA dianalisis berdasarkan *post-synthesis report* yang dihasilkan oleh Yosys [5], yang memuat statistik pemakaian *Look-Up Table* (LUT), *Flip-Flop* (FF), dan *Block RAM* (BRAM), dinyatakan dalam jumlah pemakaian pada chip FPGA Xilinx Artix-7 XC7A35T. Analisis ini turut digunakan untuk mengevaluasi efisiensi implementasi desain, termasuk perbandingan *resource utilization* antara konfigurasi kapasitas memori yang berbeda.

Biaya *resource* modul periferal UART dan I2C yang ditambahkan dievaluasi dengan dua cara. Pertama, setiap modul disintesis secara *standalone* menggunakan `synth_xilinx` pada Yosys untuk memperoleh jumlah LUT dan FF milik modul tersebut. Kedua, utilisasi keseluruhan desain setelah penambahan periferal dibandingkan dengan utilisasi konfigurasi sebelumnya. Nilai *Fmax* desain akhir juga diperiksa kembali untuk memastikan *operating frequency* 50 MHz tetap terpenuhi setelah penambahan periferal.

BAB IV
HASIL DAN PEMBAHASAN

Bab ini menguraikan hasil penelitian berupa implementasi *soft-core* RISC-V RV32I mriscv yang berjalan pada *board* FPGA Digilent Basys3, dibangun sepenuhnya dengan *open-source toolchain* (Yosys, nextpnr-xilinx, openXC7). Pembahasan mencakup keberhasilan sintesis dan implementasi di FPGA, mekanisme *program loading* lewat SPI *runtime loader*, hasil pengujian fungsional pada *hardware* fisik, verifikasi dan analisis mendalam RTL, nilai *maximum operating frequency* (*Fmax*) yang dihasilkan oleh proses *place-and-route* dan *resource utilization* pada konfigurasi operasional (RAM 32 KB, *clock core* 50 MHz), serta hasil penambahan periferal UART dan *hardware* I2C *controller*.

4.1 Keberhasilan Sintesis dan Implementasi di FPGA

RTL yang telah dimodifikasi disintesis Yosys menjadi *gate-level netlist* Xilinx Artix-7 (`synth_xilinx`). *Netlist* ini kemudian melalui *place-and-route* oleh nextpnr-xilinx ke target XC7A35T-1CPG236C mengikuti *physical constraint* `basys3_spi.xdc`, sebelum `fasm2frames` mengonversi hasilnya menjadi *configuration frame* (FASM). *Frame* tersebut akhirnya diubah `xc7frames2bit` menjadi *bitstream file* (`spi.bit`) yang diprogramkan ke FPGA lewat openFPGALoader melalui *JTAG interface*.

Proses sintesis pada konfigurasi (RAM 32 KB, *clock core* 50 MHz) menghasilkan *bitstream file* (`spi.bit`) tanpa *error* dan berhasil diprogramkan ke *board* Digilent Basys3 melalui openFPGALoader. *Firmware* diunggah lewat `cheetah_mriscv.py` dengan *output* program berikut:

```
Cheetah terbuka, SCLK = 100 kHz, mode 0, MSB-first.
Upload 42 kata dari ledshow.hex ...
Selesai. Core dilepas; program berjalan.
```

Keberhasilan sintesis dan implementasi ini dicapai menggunakan RTL final yang telah melalui serangkaian keputusan pemeriksaan dan modifikasi (Subbab 4.4).

4.2 Mekanisme *Program Loading* dan Pengembangan *Firmware*

Program dimuat ke memori *soft-core* lewat *host* Python `cheetah_mriscv.py` yang mengendalikan Total Phase Cheetah SPI Adapter. Protokol yang dipakai mengirim *frame* 66-*bit* `{instr[1:0], addr[31:0], data[31:0]}`, dikemas menjadi 9

byte (72 *bit*, dengan 6 *bit padding* di akhir yang dibuang oleh *hardware* saat CS naik), pada SPI *mode* 0 (CPOL=0, CPHA=0), MSB-*first*. Sinyal *chip-select* (CS) dipakai secara standar per *frame* (*active-low* selama transfer, sesuai konvensi SPI). Status *reset*/*release* *soft-core* (`PICORV_RST`) **tidak** dikendalikan lewat pin *reset* fisik terpisah, melainkan lewat *bit* data pada *frame* NOP yang dikirim melalui *data line* (MOSI). *Frame* NOP dengan `data=0` menahan *core* dalam kondisi *reset* selama proses penulisan program berlangsung, dan *frame* NOP dengan `data=1` di akhir *upload* melepas *core* untuk mulai mengeksekusi program. Frekuensi SCLK dibatasi maksimum ¼ *clock core*. Pada *clock core* 50 MHz (batas ∼12,5 MHz) dipakai SCLK 100 kHz, jauh di bawah batas tersebut.

Pengembangan *firmware bare-metal* menggunakan pola HAL *header-only* pada `mriscv.h` (fungsi `static inline`), yang secara terukur menghemat 169 *byte* dibandingkan pola *header*/*source* terpisah (2135 *byte* vs 2304 *byte* untuk program uji yang sama). *Library* `ssd1306.h` untuk layar OLED mendukung dua mode lewat *macro* `SSD1306_USE_BUFFER`: mode DIRECT (*default*, tiap operasi gambar langsung dikirim ke I2C, `display()` menjadi *no-op*, ∼8 *byte* RAM) dan mode BUFFER (`display()` wajib dipanggil, +1024 *byte* RAM untuk *pixel buffer*). I2C *interface* ke OLED tersedia dalam dua mode yang dipilih saat kompilasi: *software bit-bang* melalui GPIO *bank* 2 (mode *default*) dan *hardware* I2C *controller* `i2c_master.v` bila *macro* `SSD1306_HW_I2C` didefinisikan (Subbab 4.6). Pada kedua mode, urutan inisialisasi SSD1306 terdiri atas 25 *command*, termasuk `0x8D,0x14` (aktivasi *charge pump*) yang krusial agar layar tidak kosong. *Decoder rotary encoder* KY-040 membaca arah putaran pada *falling edge* sinyal CLK: `DT==1` diinterpretasikan sebagai putaran *clockwise* (CW), `DT==0` sebagai *counter-clockwise* (CCW).

Klaim awal bahwa operator pembagian (`/`, `%`) tidak dapat dipakai pada *firmware* terbukti tidak akurat. Dengan menambahkan `-lgcc` pada perintah kompilasi (`-nostdlib -lgcc`, bukan `-nostdlib` saja), operasi pembagian dapat dikompilasi dan dijalankan. Verifikasi kompilasi langsung menunjukkan rutin `__mulsi3` yang dipanggil murni berbasis *software shift-and-add*, bukan ekstensi instruksi *hardware multiplier*. Dengan demikian, ketiadaan dukungan pembagian/perkalian pada *firmware* sebelumnya merupakan persoalan konfigurasi *linker* yang belum disertakan, bukan keterbatasan arsitektur *hardware*.

Untuk perkalian, *core* juga menyediakan *hardware multiplier* `MULT.v` (Subbab 3.4.2 Bab III). Bila *firmware* dikompilasi dengan `-march=rv32i_zmmul`, operator `*` pada kode C diterjemahkan menjadi instruksi `mul` yang dieksekusi oleh `MULT.v`, bukan oleh rutin `__mulsi3`. Fungsi `MULT.v` diverifikasi dengan `mul_test.c`, yang menjalankan instruksi `mul`, `mulh`, `mulhsu`, dan `mulhu` pada 50 pasang *operand*, termasuk kasus tepi `0x80000000`, `0x7FFFFFFF`, dan `0xFFFFFFFF`. Keempat

instruksi tersebut dipanggil lewat *inline assembly*, sehingga bagian lain program tetap berupa instruksi RV32I dan tidak bergantung pada `MULT.v`. Nilai harapan dihitung di komputer *host*, dan hasilnya dilaporkan melalui UART. Seluruh 200 operasi lulus, baik pada simulasi SoC lengkap maupun pada *board* dengan *clock core* 50 MHz. Pembagian tetap tidak didukung *hardware* karena *repository* mriscv tidak menyediakan unit pembagi. Oleh karena itu, `-march=rv32im` tidak boleh dipakai, karena operator `/` dan `%` akan diterjemahkan menjadi instruksi `div`/`rem` yang memicu *trap* saat dijalankan.

4.3 Hasil Pengujian Fungsional pada *Hardware*

Seluruh sasaran pengujian fungsional berhasil dijalankan pada *hardware* fisik:

- **GPIO/LED**: `ledshow.c` dan `switch_led.c` berhasil dijalankan, dan pola nyala LED sesuai dengan program yang diunggah.
- ***Seven-segment***: `sevensegment.c` dan `encoder_7seg.c` berhasil menampilkan digit. *Byte* `gpio_datanw` dipecah menjadi dua *nibble* yang ditampilkan bergantian (*time-multiplex*) dengan periode ∼1,3 ms per digit (`scan_cnt[16]` pada *clock* 100 MHz) lewat *decode table* heksadesimal ke *seven-segment*. Saat ini tampilan terbatas pada 2 dari 4 digit yang aktif, karena *multiplexing* 4 digit belum diimplementasikan pada RTL *top module* (`an = digit_sel ? 4'b1101 : 4'b1110`).
- ***Rotary encoder* KY-040**: arah putaran (CW/CCW) yang terbaca `encoder_test.c`/`encoder_7seg.c` sesuai dengan arah putaran fisik yang dilakukan pada *encoder*.
- **Layar OLED SSD1306 (I2C *bit-bang*)**: inisialisasi dan tampilan berhasil diverifikasi lewat *firmware* pada pengujian encoder+OLED, dengan kalibrasi `ihold()` dinaikkan 4× mengikuti kenaikan *clock domain* I2C *bit-bang* dari 12,5 MHz ke 50 MHz.
- **UART**: `uart_hello.c` dan `uart_stress.c` menampilkan teks yang benar pada *serial terminal* (115200 8N1), dan `uart_echo.c` melakukan *echo* terhadap karakter yang diketik pada terminal (rincian pada Subbab 4.6.4).
- ***Hardware* I2C *Controller***: `oled_hw.c` menampilkan teks pada OLED melalui `i2c_master.v`, dan `oled_i2c_stress.c` berjalan tanpa NACK, *arbitration lost*, maupun *error* data (rincian pada Subbab 4.6.4).

Keseluruhan pengujian ini konsisten dengan kriteria analisis fungsional yang ditetapkan pada Subbab Metode Analisis Hasil Akhir, Bab III.

Keberhasilan seluruh pengujian di atas turut ditunjang oleh infrastruktur pada *board layer* (`basys3_top.v`). *Power-on reset* (POR) *circuit* diimplementasikan sebagai *counter* 16-*bit* yang menahan sinyal *reset* aktif selama ∼0,65 ms setelah *clock*

100 MHz mulai berjalan, sehingga desain dapat langsung berjalan begitu FPGA selesai dikonfigurasi tanpa memerlukan penekanan *push button reset* manual saat pengujian. Sinyal *input switch* (`sw[7:0]`) turut di-*synchronize* melalui dua *flip-flop* berurutan sebelum diteruskan ke *read path* GPIO, untuk mencegah *metastability* akibat sinyal *input* dari *board* yang bersifat *asynchronous*.

4.4 Analisis dan Verifikasi *Source Code* RTL

Soft-core mriscv dimodifikasi secara bertahap untuk memastikan kompatibilitas desain dengan *toolchain* dan *board* target, menghasilkan enam modifikasi sebagaimana ditunjukkan pada Tabel 4.1.

No	Modul	Status Modifikasi	Keterangan
1	`SP32B1024.v`	Ditulis ulang	Modul tidak tersedia (atau *missing*) pada repository mriscv
2	`axi4_interconnect.v`	Dipatch	Perubahan mekanisme latch menjadi flip-flop guna memastikan stabilitas hasil sintesis, konstruksi asli terbukti tetap dapat disintesis Yosys (Subbab 4.4.3)
3	`mriscvcore.v` + `ALU.v`	Dipatch	Penggunaan logika *tri-state* terbukti lewat sintesis penuh membuat seluruh register *write-back* di-*hardwire* menjadi nol (Subbab 4.4.5)
4	`spi_axi_master.v`	Ditulis ulang	Untuk utilisasi *SPI runtime loader* (Subbab 4.4.7)
5	`basys3_spi.xdc`	Dimodifikasi	Dimodifikasi dari `basys3_master.xdc`, berisi pengaturan *routing* dan *timing*
6	`bus_sync_sf.v`	Dipatch (152 baris)	Untuk menghindari *clock net* buatan manual lewat `~CLK`

Tabel 4.1. Ringkasan Pemeriksaan Modifikasi RTL

Rincian teknis dari modifikasi yang dilakukan diatas, termasuk cuplikan kode, metode verifikasi, serta dasar teori dari keperluan untuk melakukan modifikasi diuraikan pada subbab 4.4.1–4.4.7.

4.4.1 Modifikasi `SP32B1024.v`

Modul `SP32B1024.v` ditulis dalam bentuk RTL *behavioral* yang dipetakan ke *Block RAM* (BRAM) FPGA, karena modul dengan nama tersebut tidak tersedia pada rilis *open-source*. Modul ini menyertakan blok inisialisasi nilai awal:

```
integer i;
initial begin
    if (INIT_FILE != "") $readmemh(INIT_FILE, mem);
    else for (i = 0; i < 8192; i = i + 1) mem[i] = 32'h0;
end
```

Tanpa blok ini, isi BRAM tidak terdefinisi setelah konfigurasi Artix-7, menyebabkan kegagalan *booting* acak.

Parameter `INIT_FILE` pada modul ini mendukung dua mode operasi. **Mode operasional** (`INIT_FILE=""`, digunakan pada seluruh pengujian fungsional (Subbab 4.3)): BRAM diinisialisasi kosong (nol) saat konfigurasi FPGA, dan program dimuat setelahnya secara *runtime* lewat SPI *interface* (Subbab 4.2), sehingga proses sintesis sepenuhnya terpisah dari proses pemrograman. **Mode uji verifikasi** (`INIT_FILE` menunjuk ke *file* `.hex`, dipakai khusus untuk uji *hardware* isolasi *tri-state* (Subbab 4.4.5)): isi BRAM langsung ditentukan oleh `$readmemh` saat sintesis, sehingga *firmware* uji sudah tertanam di *bitstream* begitu FPGA dikonfigurasi, tanpa bergantung pada SPI *loader*. Mode ini mempercepat *debugging iteration cycle* karena mekanisme *program loading* dieliminasi sebagai variabel.

4.4.2 Modifikasi *Latch* pada `axi4_interconnect.v`

Pada `axi4_interconnect.v`, dua blok kombinasional dengan *sensitivity list* eksplisit, yang menghasilkan struktur *latch*, diubah menjadi struktur sekuensial *edge-triggered*:

```
// upstream (latch):
always @(m_axi_awvalid[unpk_idx] or RST or m_axi_awaddr[...])
   begin
    if (RST == 1'b0) m_axi_awaddr_g[unpk_idx] = {sword{1'b0}};
    else if (m_axi_awvalid[unpk_idx]) m_axi_awaddr_g[unpk_idx] =
    m_axi_awaddr[...];
end

// setelah dipatch:
always @(posedge CLK) begin
    if (RST == 1'b0) m_axi_awaddr_g[unpk_idx] = {sword{1'b0}};
```

```
10      else if (m_axi_awvalid[unpk_idx]) m_axi_awaddr_g[unpk_idx] =
        m_axi_awaddr[...];
11  end
```

4.4.3 *Tri-state* pada `axi4_interconnect.v`: Ditangani Otomatis oleh Yosys (TRIBUF)

Berbeda dari modifikasi *latch* pada modul yang sama (Subbab 4.4.2), `axi4_interconnect.v` juga mengandung konstruksi *tri-state* (`{sword{1'bz}}`) yang terpisah dan tetap dipertahankan tanpa dipatch, dengan parameter *default* `addressing=0` (mode *tri-state*) yang tidak di-*override* di manapun. Verifikasi lewat dua kali sintesis independen (Yosys 0.33 dan 0.38) menunjukkan hasil identik: log sintesis mencatat tahap `27.7. Executing TRIBUF pass.` dieksekusi, dan **nol** primitif TBUF muncul pada *netlist* akhir, tanpa *error*. *Warning* `"Yosys has only limited support for tri-state logic"` yang tercatat pada modul ini bersifat normal, bukan indikasi kegagalan.

Temuan ini menunjukkan Yosys (`synth_xilinx`) memiliki *pass* internal TRIBUF yang secara otomatis mengonversi pola *tri-state* **lokal sederhana** (satu `assign` per *net* dengan operator *ternary*) menjadi *multiplexer*, tanpa memerlukan RTL ditulis ulang manual. Hal ini berbeda dengan kasus pada `mriscvcore.v`/`ALU.v` (Subbab 4.4.5), yang melibatkan pola *tri-state* **lintas-modul** yang tidak tertangani otomatis oleh *pass* yang sama.

4.4.4 *Negedge* pada `AXI_SP32B1024.v`: Disengaja, *Patch* Naif Terbukti Merusak

Berbeda dari modifikasi-modifikasi sebelumnya, konstruksi `always @(negedge CLK)` pada `AXI_SP32B1024.v` sengaja **dipertahankan**, tidak dipatch:

```
1  always @(negedge CLK) begin
2      // A/CEN/WEN sengaja di-latch setengah-siklus setelah FSM
       AXI (posedge) settle
3  end
```

Kode *upstream* menyertakan komentar pengembang asli yang menjelaskan alasan penggunaan `negedge`: *"we provide the signals in negedge, because the setup and hold sh*t"*. Hipotesis awal menduga pola ini adalah praktik desain lama yang perlu dikonversi ke `posedge` agar seluruh logika bekerja pada *rising edge*.

Pengujian empiris lewat simulasi fungsional `iverilog` dengan *testbench* protokol SPI nyata membuktikan hipotesis tersebut **keliru**: mengganti `negedge`

menjadi `posedge` pada kedua baris terkait merusak fungsi *write* SRAM secara total. Penyebabnya adalah semantik *non-blocking assignment*. Setelah konversi, `SP32B1024.v` membaca *address* `A` yang tertinggal satu *clock cycle* penuh, karena hubungan *half-cycle timing* yang sebelumnya sengaja dijaga antara *state machine* AXI (*posedge*) dan modul memori (*posedge* murni) menjadi hilang.

Berdasarkan bukti ini, patch `negedge`→`posedge` **dibatalkan**, dan konstruksi `negedge` dipertahankan sebagaimana adanya. Konfirmasi *netlist* menunjukkan 47 *flip-flop negative-edge* (`FDRE_1`/`FDSE_1`) tetap ada pada hasil sintesis akhir, sesuai desain yang dipertahankan.

4.4.5 *Tri-state* pada `mriscvcore.v`/`ALU.v`: Temuan Utama yang Merusak Seluruh *Write-back Path*

Investigasi ini mengoreksi status modifikasi pada `mriscvcore.v`/`ALU.v` dari semula "opsional" menjadi **wajib**, berdasarkan bukti sintesis penuh dan dua pengujian *hardware* independen. *Write-back path* register yang semula berbentuk bus *tri-state* diubah menjadi struktur *multiplexer* berbasis operasi OR:

```
// mriscvcore.v -- setelah dipatch:
wire [31:0] rd_alu, rd_mult, rd_util, rd_mem, rd_irq;
assign rd = rd_alu | rd_mult | rd_util | rd_mem | rd_irq;

// ALU.v -- sebelum: assign rd = is_rd_nr ? OUT_Alu_rd : 32'bz;
// ALU.v -- setelah:  assign rd = is_rd_nr ? OUT_Alu_rd : 32'b0;
```

Langkah 1, sintesis Yosys saja (mode *stat*): `mriscvcore.v`/`ALU.v` dikembalikan ke versi *upstream* (*tri-state* asli), sedangkan modul lain tetap dalam kondisi terpatch. Hasil: proses selesai tanpa *error* fatal, namun log mencatat **32 warning** *"multiple conflicting drivers"*, yaitu satu per *bit* dari *bus* `rd` 32-*bit*, dengan lima modul (`ALU`, `IRQ`, `MEMORY_INTERFACE`, `MULT`, `UTILITY`) tercatat sebagai *driver* yang saling bentrok pada *net* yang sama. Total *cell* hasil sintesis 1689–1691 (dibandingkan 3996 pada versi terpatch), mengindikasikan banyak logika yang gugur karena resolusi konflik.

Langkah 2, sintesis penuh via `make synth`: *Warning* berubah nama menjadi *"Driver-driver conflict"*, dengan total **128 warning** (32 *bit* × 4 modul: `ALU`, `MEMORY_INTERFACE`, `MULT`, `UTILITY`). `IRQ` tidak muncul pada pencarian ini, kemungkinan disebabkan tahap *pipeline* sintesis yang berbeda, dan hal ini belum diverifikasi tuntas. Cuplikan aktual dari log sintesis:

```
Warning: Driver-driver conflict for
\u_impl.mriscvcore_inst.ALU_inst.OUT_Alu_rd[31]
```

```
between ALU_inst.$procdff$6064.Q
and constant 1'0 in basys3_top:
Resolved using constant.
```

Seluruh 128 konflik (128/128) diselesaikan Yosys dengan cara yang identik: **`Resolved using constant`**, bernilai `1'0`. Artinya *net* `rd` pada *netlist* akhir pada dasarnya **selalu bernilai nol**, apa pun instruksi yang dieksekusi.

Kenapa ini merusak seluruh eksekusi, bukan hanya `lw`: Pada `mriscvcore.v`, `rd` bukan sekadar *output path* dari `MEMORY_INTERFACE` (operasi *load*), melainkan **satu-satunya *write-back bus*** menuju `REG_FILE`, yang dipakai bersama oleh seluruh modul yang menghasilkan nilai untuk ditulis ke register (`ALU`, `MEMORY_INTERFACE`, `MULT`, `IRQ`, `UTILITY`). Karena `rd` di-*hardwire* ke nol, **setiap instruksi yang menulis register, termasuk operasi aritmetika biasa seperti `add/addi/li`, juga menghasilkan nilai nol**, bukan hanya instruksi `lw`.

Uji *hardware* dengan dua *firmware* independen, program di-*bake* langsung ke *bitstream* lewat `INIT_FILE`:

Firmware 1 (uji *write-read* RAM): `sw` nilai `0xAA` ke *address* RAM `0x400`, `lw` kembali, lalu tampilkan hasilnya ke LED lewat `led_set()`. **Hasil: seluruh LED mati** (ekspektasi: pola `10101010`).

Firmware 2 (`test_tristate.c`, uji kontrol yang murni *write* tanpa `lw` sama sekali):

```c
#include "mriscv.h"
int main(void) {
    led_set(0xAA);    /* langsung tulis konstanta ke LED, TANPA
   sw+lw RAM */
    for(;;){}
}
```

Hasil: seluruh LED mati juga. *Firmware* ini semula dirancang untuk mengisolasi apakah kegagalan spesifik pada *path* `lw`. Namun karena instruksi `li`/`addi` (dipakai `led_set()`) **juga** melewati *bus* `rd` yang sama untuk *write-back* ALU, desain uji kontrol ini tidak benar-benar mengisolasi variabel seperti yang dimaksudkan. Kelemahan ini ditemukan lewat analisis log sintesis, bukan lewat uji *hardware* tambahan.

Verifikasi mekanisme `INIT_FILE` bekerja (menyingkirkan kemungkinan alternatif): Parameter `INIT_FILE` dibaca langsung dari *netlist* hasil sintesis dalam bentuk 136-*bit* biner, didekode manual 8-*bit* per karakter, menghasilkan persis string `"test_tristate.hex"`:

```
Parameter INIT_FILE (136 bit):
01110100011001010111001101110100 01
01111101110100011100100110100101 11
00110111010001100001011101000110 01
01001011100110100001100101011110 00
-> didekode 8-bit/karakter -> "test_tristate.hex"
```

Hasil ini membuktikan mekanisme *firmware loading* langsung ke *bitstream* (tanpa SPI *loader*) bekerja dengan benar. Kegagalan LED pada kedua *firmware* uji **bukan** disebabkan kegagalan *program loading*, melainkan murni akibat bug *tri-state* pada *write-back path*.

Kesimpulan: Modifikasi pada `mriscvcore.v`/`ALU.v` (*tri-state* $\rightarrow$ OR-*mux*) bersifat **wajib**, bukan opsional. Berbeda dari kasus `axi4_interconnect.v` (Subbab 4.4.3), di mana *pass* TRIBUF menangani *tri-state* otomatis dengan sempurna, pola *tri-state* **lintas-modul** (banyak instansiasi modul terpisah berbagi satu *wire* lewat *port*, bukan satu `assign` lokal) tidak tertangani oleh *pass* yang sama. Yosys justru menyelesaikannya secara diam-diam menjadi konstanta nol, sebuah *silent failure* yang lolos dari `CHECK pass` Yosys. `CHECK pass` melaporkan `"0 problems"` karena memang tidak memeriksa *driver-driver conflict*, sehingga hal ini merupakan perbedaan cakupan pemeriksaan, bukan kontradiksi.

Status pengujian lanjutan: pengujian *positive control* (memprogram ulang versi RTL yang sudah dipatch/OR-*mux* dengan *firmware* uji yang sama, memverifikasi LED menyala pola `10101010`) direkomendasikan sebagai langkah verifikasi penutup, namun hasilnya belum dilaporkan secara terpisah pada penelitian ini.

4.4.6 Parameter `addr_mask`/`addr_use` Tidak Ter-*override*: Ekspansi RAM dan Relokasi Periferal Sempat Mengalami *Silent Failure*

Temuan verifikasi lain muncul pada tahap instansiasi `axi4_interconnect.v` di dalam `impl_axi.v`. Modul `axi4_interconnect.v` memiliki parameter `addr_mask`/`addr_use` sendiri dengan nilai *default* yang di-*hardcode* (mengarah ke *address map* RAM 4 KB lama), sedangkan `impl_axi.v` turut mendefinisikan `localparam` bernama sama berisi *address map* baru (RAM 32 KB). Namun, keduanya berada pada *scope* modul yang berbeda. Karena instansiasi `axi4_interconnect.v` pada `impl_axi.v` awalnya dilakukan tanpa *override* parameter secara eksplisit (`#(.addr_mask(addr_mask), .addr_use(addr_use))`), `localparam` pada `impl_axi.v` tidak pernah benar-benar dipakai. *Decoder* AXI tetap diam-diam berjalan dengan *address map* 4 KB lama, walaupun *file* RTL "kelihatannya" sudah diperbarui.

Akibatnya, ekspansi RAM ke 32 KB serta relokasi periferal DAC/ADC/GPIO ke *address* `0x4000+` tampak tidak berpengaruh sama sekali secara fungsional, tanpa disertai kegagalan atau *hang* yang jelas. Kondisi ini sulit dicurigai lewat inspeksi kode maupun dari log sintesis, karena sintesis tetap berhasil tanpa *warning* tambahan dan RTL tetap valid secara sintaksis. Bug ini baru ditemukan lewat simulasi fungsional `iverilog`, bukan dari sintesis maupun dari observasi hasil pada *hardware*. Perbaikannya adalah mendeklarasikan *override* parameter secara eksplisit pada setiap instansiasi modul yang memiliki parameter *address*/konfigurasi bernama sama dengan `localparam` pada modul *parent*-nya.

Temuan ini menegaskan prinsip metodologis yang sama (Subbab 4.4.3–4.4.5): klaim bahwa suatu modifikasi RTL "sudah diterapkan" tidak dapat diandalkan hanya dari keberhasilan sintesis atau inspeksi *source* sepintas. Verifikasi lewat simulasi fungsional maupun pengujian *hardware* tetap diperlukan untuk memastikan perubahan benar-benar berpengaruh terhadap perilaku desain.

4.4.7 Multi-*clock* ke Single-*clock* pada `spi_axi_master.v`

Modul `spi_axi_master.v` pada rilis *open-source* mriscv (445 baris) dirancang dengan dua *clock domain* berbeda: tujuh blok `always` yang di-*trigger* oleh `posedge SCLK` (mengikuti *clock* SPI eksternal dari *host*) dan dua blok `always` yang di-*trigger* oleh `posedge CLK` (mengikuti *clock core* internal). Desain dua *clock domain* semacam ini lazim ditangani lewat mekanisme *clock domain crossing* (CDC) eksplisit, namun tidak sesuai dengan arsitektur SPI *runtime loader* pada penelitian ini, yang mensyaratkan `spi_axi_master.v` beroperasi sepenuhnya pada satu *clock domain* (*clock core*) agar dapat bertindak sebagai *master* bus AXI4-Lite yang sejajar dengan `mriscvcore.v` tanpa mekanisme sinkronisasi CDC tambahan.

Modul ini ditulis ulang menjadi 101 baris, seluruhnya di-*trigger* oleh `posedge CLK` (tiga blok `always`), dengan sinyal SCLK eksternal dideteksi lewat teknik *oversampling* alih-alih dijadikan sumber *clock* langsung. Modifikasi ini bersifat **wajib** untuk arsitektur SPI *runtime loader* yang diterapkan pada penelitian ini, bukan sekadar pilihan gaya penulisan RTL.

4.5 *Maximum Operating Frequency* (*Fmax*) Hasil *Place-and-Route* dan *Resource Utilization*

4.5.1 Eksperimen A: *Repeatability*

Proses `make synth` dijalankan lima kali berturut-turut per konfigurasi, tanpa mengubah RTL di antara *run* (`make clean` dijalankan sebelum tiap *run* agar Yosys/nextpnr benar-benar dieksekusi ulang, bukan dilewati oleh *dependency cache* Make). Hasil ditunjukkan pada Tabel 4.2.

Konfigurasi	***n***	***Fmax* Rata-rata (MHz)**	**Std. Dev. (MHz)**
RAM 4 KB, *clock* 1,5625 MHz	5	66,16	0,00
RAM 32 KB, *clock* 50 MHz	5	67,57	0,00

Tabel 4.2. Hasil Eksperimen *Repeatability* Sintesis ($n = 5$ per konfigurasi)

Standar deviasi bernilai nol pada kedua konfigurasi karena kelima *run* per konfigurasi menghasilkan *Fmax* yang identik *bit*-demi-*bit*. Hal ini menunjukkan `nextpnr-xilinx` bersifat **deterministik** untuk RTL dan *constraint* yang sama ketika memakai nilai *seed default* internalnya (bukan *seed* eksplisit yang diacak), sehingga sintesis berulang pada RTL tak berubah selalu mereproduksi hasil *place-and-route* yang identik persis.

Resource utilization turut identik pada kelima *run* per konfigurasi, sebagaimana diharapkan karena RTL yang disintesis sama. Hasilnya disajikan pada Tabel 4.3.

Konfigurasi	**LUT**	**FF**	**RAMB36E1**	**DSP48**	**CARRY4**
RAM 4 KB, 1,5625 MHz	2163	1395	1	0	225
RAM 32 KB, 50 MHz	2181	1403	8	0	223

Tabel 4.3. *Resource Utilization* FPGA (Xilinx Artix-7 XC7A35T)

Utilisasi RAMB36E1 naik dari 1 menjadi 8 *block*, konsisten dengan ekspansi kapasitas SRAM 4 KB→32 KB (kenaikan 8×). Utilisasi LUT, FF, dan CARRY4 relatif hampir sama antarkonfigurasi (selisih $< 1\%$), karena perubahan RTL utama (lebar *address bus* 10→13 *bit*, relokasi *address map* AXI) hanya menyentuh sedikit logika kombinasional, bukan menambah blok fungsional besar. **DSP48 bernilai 0 pada kedua konfigurasi.** Nilai ini tidak berarti `MULT.v` absen dari desain. `MULT.v` diimplementasikan sebagai *Booth multiplier* iteratif yang hanya memakai operasi penjumlahan dan *shift*, tanpa operator perkalian Verilog, sehingga Yosys memetakannya ke LUT, FF, dan CARRY4, bukan ke DSP48. Modul ini tetap tersintesis karena terhubung ke *control path* dan *write-back path* pada `mriscvcore.v` (Subbab 3.4.2 Bab III), dan *cell* milik `MULT_inst` tercatat pada log sintesis kedua konfigurasi. Sintesis *standalone* `MULT.v` menggunakan `synth_xilinx` menghasilkan 651 LUT, 248 FF, 103 CARRY4, dan 0 DSP48, atau setara sekitar 30% dari total LUT konfigurasi RAM 32 KB. Verifikasi fungsional `MULT.v` dibahas pada Subbab 4.2.

4.5.2 Eksperimen B: Sensitivitas *Seed*

RTL yang disintesis pada eksperimen ini tidak diubah dari konfigurasi RAM 32 KB/50 MHz (sama dengan Eksperimen A). *Netlist* Yosys (`spi.json`) hanya dihasilkan satu kali dan dipakai ulang untuk seluruh 10 *run* `nextpnr-xilinx` dengan

nilai `-seed` eksplisit berbeda-beda, sehingga perbedaan hasil murni berasal dari algoritma *placement-and-routing*, bukan variasi *netlist*. Hasil ditunjukkan pada Tabel 4.4.

Seed	***Fmax* (MHz)**	***Seed***	***Fmax* (MHz)**
1	69,88	6	73,31
2	68,74	7	69,48
3	70,12	8	68,97
4	69,95	9	66,37
5	63,59	10	67,01

Tabel 4.4. Sensitivitas *Fmax* terhadap Nilai *Seed* `nextpnr-xilinx` (RAM 32 KB, 50 MHz)

Rata-rata *Fmax* dari 10 *seed* tersebut adalah 68,74 MHz dengan standar deviasi 2,61 MHz (≈3,8% dari rata-rata), rentang 63,59–73,31 MHz (selisih 9,72 MHz, ≈14,1% dari rata-rata). Hasil ini menegaskan bahwa algoritma *placer* (*simulated annealing*) pada `nextpnr-xilinx` memang bersifat stokastik terhadap nilai *seed* eksplisit. Kondisi ini berbeda dari Eksperimen A yang memakai *seed default* tetap [4]. Nilai *Fmax* pada konfigurasi *seed default* (67,57 MHz, Tabel 4.2) berada di dalam rentang hasil *sweep* 10-*seed* ini dan dekat dengan mediannya, menunjukkan bahwa *seed default* bukan kasus kebetulan terbaik atau terburuk, melainkan representasi yang cukup tipikal.

Clock core beroperasi pada 50 MHz. Bahkan pada *seed* terburuk dari 10 sampel (*seed* 5, 63,59 MHz), *Fmax* masih ≈1,27× di atas *operating point* 50 MHz. *Safety margin* desain berkisar 1,27×–1,47× di seluruh 10 *seed* yang diuji, dan tidak ada *seed* dalam sampel ini yang menyebabkan *Fmax* turun di bawah 50 MHz. Hasil ini bukan jaminan formal atas keseluruhan ruang nilai *seed* (10 *seed* adalah sampel, bukan populasi lengkap), namun memberikan bukti empiris tambahan bahwa desain saat ini memiliki margin yang cukup terhadap variasi hasil *place-and-route*, bukan hanya kebetulan dari satu hasil *seed default* saja.

4.6 Penambahan Periferal UART dan I2C *Controller*

Subbab ini menguraikan hasil penambahan tiga modul periferal yang dirancang pada Subbab 3.4.4 Bab III, yaitu `uart_tx.v`, `uart_rx.v`, dan `i2c_master.v`, beserta GPIO *bank* 2. Pembahasan mencakup integrasi ke bus AXI4-Lite, detail implementasi, hasil verifikasi pada simulasi dan *hardware*, temuan selama integrasi, serta dampak penambahan periferal terhadap *resource utilization* dan *Fmax*.

4.6.1 Integrasi ke Bus AXI4-Lite

Keempat *slave* baru diintegrasikan pada `impl_axi.v` dengan menaikkan parameter `slaves` dari 5 menjadi 9 dan memperluas `addr_mask`/`addr_use`. Elemen paling kiri pada kedua vektor tersebut berhubungan dengan *slave* berindeks tertinggi:

```
localparam slaves = 9;
localparam [slaves*sword-1:0] addr_mask = {32'h00000003, 32'
   h0000000F,
    32'h00000000, 32'h00000000, 32'h00000000, 32'h0000000F,
    32'h00000001, 32'h00000001, 32'h00001FFF};
localparam [slaves*sword-1:0] addr_use  = {32'h00004040, 32'
   h00004030,
    32'h00004021, 32'h00004020, 32'h04000000, 32'h00004010,
    32'h00004008, 32'h00004000, 32'h00000000};
```

UART TX dan UART RX masing-masing hanya memiliki satu register sehingga memakai *mask* `0x0` (*exact match*). GPIO *bank* 2 memakai *mask* `0xF` seperti GPIO *bank* 1, sedangkan I2C *controller* memakai *mask* `0x3` untuk empat register.

Ketika `slaves` pertama kali dinaikkan menjadi 6 untuk UART TX, `iverilog` menampilkan *warning port width mismatch* pada instansiasi `axi4_interconnect.v`. Instansiasi tersebut ternyata hanya meng-*override* `addr_mask` dan `addr_use`, sedangkan `masters` dan `slaves` tetap memakai nilai *default* di dalam `axi4_interconnect.v` (`masters = 2`, `slaves = 5`). Selama desain memang memiliki dua *master* dan lima *slave*, nilai *default* tersebut kebetulan benar sehingga kekurangan ini tidak terlihat. Temuan ini merupakan kasus kedua dari kelas bug yang sama dengan Subbab 4.4.6, yaitu parameter *child module* yang diam-diam memakai nilai *default*. Perbaikannya adalah meng-*override* seluruh parameter secara eksplisit:

```
axi4_interconnect #(
    .masters(masters),
    .slaves(slaves),
    .addr_mask(addr_mask),
    .addr_use(addr_use)
) inst_axi4_interconnect ( ... );
```

4.6.2 Implementasi UART

Pada `uart_tx.v`, penulisan saat *transmitter busy* tetap diterima oleh *write channel* AXI, tetapi *byte*-nya di-*drop* (Subbab 3.4.4 Bab III). Pengujian semantik

non-blocking ini dengan *testbench* menemukan *race condition*: terdapat *window* satu *clock cycle* antara saat pulsa `start_pulse` dibangkitkan dan saat FSM *transmitter* benar-benar meninggalkan *state* IDLE. Penulisan kedua yang tiba tepat pada *window* tersebut masih melihat *transmitter* tidak *busy*, sehingga dapat menimpa *byte* pertama sebelum *byte* itu mulai dikirim. Perbaikannya adalah memasukkan `start_pulse` ke dalam definisi *busy*:

```
wire busy = (tstate != S_IDLE) || start_pulse;
```

Pada `uart_rx.v` ditemukan *race* serupa antara pembacaan register dan penerimaan *byte*. Pada versi awal, *flag valid* di-*clear* satu *clock cycle* setelah isi register di-*snapshot* ke `RDATA`. Akibatnya, *byte* yang selesai diterima tepat pada *cycle snapshot* tidak ikut ter-*snapshot*, lalu *flag valid*-nya ikut ter-*clear*, sehingga *byte* tersebut hilang tanpa menyalakan *flag overrun*. Pada versi akhir, *pop* terjadi pada *cycle* yang sama dengan *snapshot*, dan *byte* baru yang tiba pada *cycle* itu diprioritaskan atas *pop*:

```
assign rd_pop = RST && (ar_state == AR_IDLE) && ARVALID;
always @(posedge CLK) begin
    ...
    if (rd_pop) begin rx_valid <= 0; rx_ovr <= 0; rx_ferr <= 0;
   end
    if (done) begin                      // byte baru menang atas
   pop
        rx_data  <= done_d;
        rx_valid <= 1'b1;
        if (rx_valid && !rd_pop) rx_ovr <= 1'b1;
        if (done_fe) rx_ferr <= 1'b1;
    end
end
```

Kasus ini diuji pada `uart_rx_tb.v` dengan melakukan *polling* register secara terus-menerus sambil menerima 8 *byte* berurutan, diulang untuk 12 nilai *offset* awal agar saat pembacaan jatuh pada semua *phase* relatif terhadap akhir penerimaan *byte*.

Di sisi *firmware*, *polling* hingga *transmitter* siap dilakukan oleh fungsi HAL pada `mriscv.h`. Penerimaan disediakan dalam versi *non-blocking* (`uart_getc_nb()`, mengembalikan -1 bila belum ada data) dan versi *blocking* (`uart_getc()`):

```
#define UART_TXD (*(volatile unsigned *)(0x10080u))
#define UART_RXR (*(volatile unsigned *)(0x10084u))

static inline unsigned uart_busy(void){ return UART_TXD & 1u; }
static inline void uart_putc(char c){
```

```
    while (uart_busy()) {}                  /* polling bit busy */
    UART_TXD = (unsigned)(unsigned char)c;
}
static inline void uart_puts(const char *s){ while (*s) uart_putc(*s++); }

static inline int uart_getc_nb(void){
    unsigned v = UART_RXR;                  /* baca = pop */
    return (v & 0x100u) ? (int)(v & 0xFFu) : -1;
}
```

4.6.3 Implementasi I2C *Controller*

Pada *top module* `basys3_top.v`, port `oled_scl` dan `oled_sda` diubah dari `output` menjadi `inout` agar level *line* dapat dibaca kembali oleh *controller* untuk keperluan ACK, pembacaan data, *clock stretching*, dan deteksi *arbitration lost*. Logika *wired-AND* antara GPIO *bank* 2 dan `i2c_master.v` dituliskan sebagai berikut:

```
assign oled_scl = (gpio2_datanw[0] & i2c_scl_o) ? 1'bz : 1'b0;
assign oled_sda = (gpio2_datanw[1] & i2c_sda_o) ? 1'bz : 1'b0;
```

Kedua pin memakai *pull-up* internal melalui atribut `PULLUP true` pada *file* XDC. Karena GPIO *bank* 2 bernilai nol setelah *reset*, fungsi `i2c_hw_init()` menulis logika 1 ke kedua pin GPIO tersebut sebelum *hardware controller* dipakai. Pengiriman satu *byte* dilakukan oleh fungsi HAL berikut, yang mengembalikan nilai 1 bila *slave* menjawab ACK:

```
static inline int i2c_hw_write(unsigned char b, unsigned flags){
    I2C_DATA = b;
    I2C_CMD  = I2C_WRITE | flags;           /* flags: I2C_START, I2C_STOP */
    return (i2c__wait() & 6u) == 0;         /* bit1 NACK, bit2 arb_lost */
}
```

Dengan nilai DIV *default* 31, satu periode SCL sama dengan $4 \times 31 = 124$ *clock cycle*, atau 403,2 kHz pada *clock core* 50 MHz. Nilai ini 0,8% di atas batas 400 kHz *Fast-mode*, karena hasil $50\,000\,000/(4 \times 400\,000) = 31{,}25$ dibulatkan ke bawah menjadi 31. Layar SSD1306 bekerja normal pada frekuensi ini. Bila kepatuhan ketat terhadap batas *Fast-mode* diperlukan, *firmware* cukup menulis DIV $= 32$ sehingga SCL menjadi 390,6 kHz.

Pengiriman satu *byte* beserta *bit* ACK membutuhkan 9 periode SCL, yaitu $9 \times 124 = 1.116$ *clock cycle*. Pada simulasi, *firmware* yang mengirim setiap *byte* dengan urutan *write* DATA, *write* CMD, lalu *polling* STATUS menambahkan sekitar 290 *clock cycle* per *byte*, sehingga *controller* berada dalam kondisi *busy* sekitar 79% dari waktu selama transfer *back-to-back*.

4.6.4 Hasil Verifikasi Simulasi dan *Hardware*

Hasil verifikasi tiga *level* yang dijelaskan pada Subbab 3.4.4 Bab III dirangkum pada Tabel 4.5.

Level	***Testbench / Firmware***	**Skenario yang Diperiksa**	**Hasil**
Unit	`uart_tx_tb.v`	Pengiriman `0xA5` dan `0x00`, TXD kembali *idle high*, *write* saat *busy* selesai cepat dan *byte*-nya di-*drop* tanpa transmisi susulan	PASS
Unit	`uart_rx_tb.v`	Penerimaan *byte*, *flag valid*, *overrun*, dan *framing error*, *glitch*, *back-to-back byte*, *read/receive race* (12 *offset* $\times$ 8 *byte*)	PASS
Unit	`i2c_master_tb.v`	*Write* 3 *byte*, NACK pada *address* salah, *read* dengan ACK/NACK, *repeated* START, *clock stretching*, DIV < 4 ditolak, *arbitration lost*	PASS
SoC	`impl_axi_uartrx_tb.v` + `uart_echo.c`	*Banner* terkirim, karakter dari *testbench* di-*echo*, statistik `ovr=0 ferr=0`, tanpa *trap*	PASS
SoC	`basys3_top_i2c_tb.v` + `i2c_test.c`	Urutan *write*, NACK, *repeated* START, dan *read* terhadap model *slave* I2C. LED $=$ `0x5A`, tanpa *trap*	PASS
Board	`uart_hello.c`	Teks berulang tampil benar pada *serial terminal* 115200 8N1	Berhasil
Board	`uart_stress.c`	3202 baris dalam 20 detik tanpa baris *corrupt* dan tanpa nilai *counter* yang terlewat	Berhasil
Board	`uart_echo.c`	Karakter yang diketik pada terminal di-*echo*	Berhasil
Board	`oled_hw.c`	Teks "Hello World" tampil melalui *hardware* I2C *controller*	Berhasil
Board	`oled_i2c_stress.c`	Ronde pola layar berulang pada DIV 31, 62, dan 125. *Error counter* N, A, S, T tetap nol	Berhasil
Board	`oled_stress.c`	Regresi I2C *bit-bang* setelah *line* diubah menjadi *wired-AND*	Berhasil

Tabel 4.5. Ringkasan Verifikasi Modul UART dan I2C

Uji `uart_stress.c` mengirim baris sepanjang 66 *byte*, berisi *counter* heksadesimal 8 digit dan sebuah kalimat tetap, secara terus-menerus tanpa jeda. *String*

dibaca dari RAM saat *runtime*, sehingga uji ini juga melibatkan *load/store path* pada *core*. Dalam 20 detik diterima 3202 baris, setara dengan $3202 \times 66/20 \approx 10.570$ *byte*/s, atau sekitar 92% dari *throughput* maksimum teoretis 11.520 *byte*/s pada 115200 *baud* 8N1 (10 *bit* per *byte*). Tidak ditemukan baris yang *corrupt* maupun nilai *counter* yang terlewat.

Uji `oled_i2c_stress.c` memakai mode BUFFER dan menjalankan ronde berulang tanpa henti. Setiap ronde berisi layar putih penuh dan hitam penuh (sekitar 1.044 *byte* per *frame*), 16 *frame* batang vertikal yang bergeser, pola *checkerboard*, layar teks status, serta 512 kali perubahan kontras layar yang masing-masing terdiri atas dua transaksi pendek (START, tiga *byte*, STOP), sehingga terdapat 1.024 transaksi berurutan. Nilai DIV diganti setiap ronde secara bergiliran: 31, 62, dan 125, atau sekitar 400, 200, dan 100 kHz. *Firmware* memeriksa sendiri empat jenis *error* dan melaporkannya melalui UART setiap ronde: N (*byte* yang dijawab NACK), A (*arbitration lost*), S (*checksum* isi *buffer* yang tidak konsisten), dan T (waktu pengiriman *frame*, diukur dengan `rdcycle`, berada di luar *window* teoretis, yang menandakan *bit* hilang atau *controller* macet). Selama pengujian keempat *counter* tetap bernilai nol. Uji ini sekaligus memanfaatkan UART sebagai *channel* pelaporan, sehingga kedua periferal baru bekerja bersamaan.

4.6.5 Temuan: *Hang* pada `uart_puts()` Disebabkan *Linker Script*

Pada tahap awal pengujian UART di *board*, sebagian *firmware* berhenti total tanpa mengirim satu karakter pun. Gejalanya tampak bergantung pada panjang *string*: `uart_puts()` dengan *string* tiga karakter berjalan normal, sedangkan *string* empat karakter langsung berhenti, padahal hasil *disassembly* kedua program identik kecuali isi *string*-nya. Dugaan awal mengarah ke *hardware*, antara lain perilaku *read-during-write* pada *register file* hasil sintesis, *counter* kecil di dalam `axi4_interconnect.v`, dan margin *timing*. Dugaan *timing* gugur karena gejala yang sama tetap muncul ketika *clock core* diturunkan ke 25 MHz.

Penyebab sebenarnya ada pada *linker script* `link_c.ld`. *Directive* `. = ALIGN(4)` yang ditulis di luar *section* `.bss` tidak menjamin `.bss` dimulai pada *address* kelipatan 4. Ketika panjang `.rodata` bukan kelipatan 4, misalnya *string* `"0123"` beserta *null terminator* (5 *byte*), `__bss_start` jatuh pada *address* `0x105`. *Startup code* `crt0.S` kemudian mengosongkan `.bss` dengan instruksi `sw`, yang pada *address* tersebut menjadi akses *misaligned*, sehingga FSM *core* masuk *state trap* sebelum `main()` sempat dijalankan. *String* tiga karakter kebetulan aman karena panjangnya beserta *null terminator* tepat 4 *byte*. Perbaikannya adalah meletakkan *alignment* di dalam definisi *section*:

```
/* sebelum */  .bss : {
```

```
/* sesudah */  .bss : ALIGN(4) {
```

Hubungan sebab-akibat ini dikonfirmasi pada *board*: *firmware* uji dengan kode instruksi identik berhenti ketika di-*link* dengan *linker script* lama dan berjalan normal dengan *linker script* yang telah diperbaiki. Daftar *firmware* yang *address* `.bss`-nya tidak *aligned* pada *linker script* lama juga tepat sama dengan daftar *firmware* yang sebelumnya tercatat berhenti. Seperti kasus `negedge` pada Subbab 4.4.4, dugaan awal yang keliru tetap didokumentasikan karena menunjukkan bahwa gejala yang tampak seperti kerusakan *hardware* dapat berasal dari *software* pendukung.

Temuan ini mendorong dua perbaikan tambahan pada RTL. Pertama, sinyal `trap` dari *core* disambungkan ke LED LD15, karena tanpa indikator tersebut kondisi *trap* dan kondisi bus yang macet (*stuck*) sama-sama hanya terlihat sebagai LED yang berhenti. Kedua, `impl_axi.v` diberi sinyal *bus reset* (`BUS_RST`) yang membangkitkan pulsa *reset* singkat pada `axi4_interconnect.v` dan seluruh *slave* setiap kali *core* masuk kondisi *reset*. Sebelum perbaikan ini, *trap* yang terjadi di tengah transaksi *write* dapat membuat *interconnect* terus menunggu *handshake* dari *core* yang sudah berhenti, sehingga seluruh *upload* SPI berikutnya gagal tanpa pesan *error* sampai FPGA di-*reset* penuh. Perbaikan ini diverifikasi dengan simulasi (`impl_axi_busrst_tb.v`) dan pada *board*.

4.6.6 *Resource Utilization* dan *Fmax* Setelah Penambahan Periferal

Tabel 4.6 menunjukkan hasil sintesis *standalone* setiap modul menggunakan `synth_xilinx` pada Yosys 0.38. Jumlah LUT dihitung dari *cell* LUT1–LUT6 pada laporan `stat`, dengan cara yang sama seperti Tabel 4.3.

Modul	**LUT**	**FF**	**CARRY4**
`uart_tx.v`	41	39	4
`uart_rx.v`	39	62	4
`i2c_master.v`	95	100	10
Total	**175**	**201**	**18**

Tabel 4.6. *Resource Utilization* Modul UART dan I2C (Sintesis *Standalone*)

Perbandingan utilisasi keseluruhan desain sebelum dan sesudah penambahan periferal ditunjukkan pada Tabel 4.7. Baris pertama diambil dari Tabel 4.3, sedangkan baris kedua diambil dari laporan `stat` sintesis desain akhir.

Konfigurasi	**LUT**	**FF**	**RAMB36E1**	**DSP48**	**CARRY4**
Sebelum penambahan	2181	1403	8	0	223
Desain akhir	2532	1636	8	0	241
Selisih	+351	+233	0	0	+18

Tabel 4.7. *Resource Utilization* Sebelum dan Sesudah Penambahan Periferal (RAM 32 KB, 50 MHz)

Kenaikan total adalah 351 LUT ($+16\%$) dan 233 FF ($+17\%$). Berdasarkan sintesis *standalone*, ketiga modul periferal menyumbang 175 LUT dan 201 FF. Sisanya, sekitar 176 LUT dan 32 FF, berasal dari perubahan lain yang masuk ke RTL pada periode yang sama dan tidak diukur secara terpisah, antara lain perluasan logika *decoder* dan *multiplexer* pada `axi4_interconnect.v` dari lima menjadi sembilan *slave*, instansiasi kedua `completogpio.v`, *reset synchronizer* pada *top module*, serta logika *bus reset*. Kenaikan CARRY4 sebesar 18 sama dengan jumlah CARRY4 ketiga modul periferal. Kebutuhan BRAM dan DSP48 tidak berubah, dan desain akhir hanya memakai sekitar 12% dari 20.800 LUT pada XC7A35T, sehingga ruang untuk periferal tambahan masih besar.

Desain akhir di-*place-and-route* dengan target `--freq 50` dan *seed* 13, menghasilkan *Fmax* 60,87 MHz (`PASS at 50.00 MHz`), atau margin 1,22$\times$ terhadap *clock core*. Laporan pertama nextpnr-xilinx pada *run* yang sama mencatat 47,28 MHz (`FAIL`), tetapi angka tersebut merupakan estimasi sebelum *routing*. Sesuai Subbab 3.5.2 Bab III, nilai yang dipakai adalah laporan kedua. *Seed* 13 dipilih dari *sweep* 20 *seed* pada konfigurasi antara (UART TX/RX dan GPIO *bank* 2, sebelum I2C ditambahkan) yang menghasilkan rata-rata 60,16 MHz, standar deviasi 3,16 MHz, dan rentang 55,74–66,28 MHz, dengan seluruh *seed* memenuhi target 50 MHz. Pada konfigurasi antara tersebut *seed* 13 memberi nilai tertinggi (66,28 MHz), sedangkan setelah I2C ditambahkan *seed* yang sama memberi 60,87 MHz. Artinya, *seed* terbaik untuk satu *netlist* tidak otomatis tetap terbaik setelah RTL berubah, sehingga *sweep* perlu diulang setiap kali RTL diubah. *Sweep seed* pada desain akhir belum dilakukan pada penelitian ini.

Nilai *Fmax* pada subbab ini tidak dapat dibandingkan langsung dengan Eksperimen A dan B (Subbab 4.5). Eksperimen A dan B dijalankan dengan target *default* nextpnr-xilinx (log mencatat `PASS at 12.00 MHz`), sedangkan desain akhir dijalankan dengan target eksplisit 50 MHz, sehingga sasaran optimasi *placer* pada kedua kelompok eksperimen berbeda.

Beberapa keterbatasan periferal yang ditambahkan perlu dicatat. *Baud rate*

UART bersifat tetap (*fixed*) dan hanya dapat diubah melalui parameter saat sintesis. UART *receiver* hanya memiliki *holding register* satu *byte* tanpa FIFO, sehingga *firmware* yang lama tidak membaca register dapat kehilangan data (ditandai *flag overrun*). Selain itu, *firmware* `encoder_oled.c`, yang memerlukan *rotary encoder* fisik, belum diuji ulang di *board* setelah *line* I2C diubah menjadi *wired-AND*.

4.7 Perbandingan dengan Penelitian Terdahulu

Sebagai pembanding, tim pengembang mriscv sendiri (`onchipuis`) telah mengimplementasikan *soft-core* ini pada FPGA lewat *repository* `mriscv_vivado`, menggunakan *board* Nexys4 DDR dengan chip Xilinx Artix-7 XC7A100T dan *toolchain* Vivado 2016.4 (*proprietary*). Perbandingan karakteristik implementasi kedua penelitian ditunjukkan pada Tabel 4.8.

Aspek	**Penelitian Ini**	**`mriscv_vivado`**
FPGA	Basys3, XC7A35T	Nexys4 DDR, XC7A100T (3× lebih besar)
Toolchain	Sepenuhnya *open-source* (Yosys, nextpnr-xilinx, openXC7)	Vivado *proprietary*
Memori	BRAM 32 KB	BRAM + DDR2 eksternal
Program loading	SPI *runtime loader* (independen dari *bitstream*)	`write_cfgmem` (digabung dalam satu *file* `.mcs`)
Periferal	GPIO (2 *bank*), UART TX/RX, *hardware* I2C *controller*, OLED, *rotary encoder*	+UART, xADC, DAC fungsional, DDR2
Linker	`-nostdlib` (*default*). `-lgcc` terverifikasi dapat ditambahkan untuk operasi pembagian	`-nostdlib -lgcc`

Tabel 4.8. Perbandingan Karakteristik Implementasi dengan `mriscv_vivado`

Perbedaan paling signifikan terletak pada mekanisme *program loading*. `mriscv_vivado` menggabungkan *bitstream* konfigurasi FPGA dan program *firmware* ke dalam satu *file* `.mcs` lewat `write_cfgmem`, sehingga setiap perubahan *firmware* menuntut regenerasi ulang seluruh *configuration file*. Arsitektur SPI *runtime loader* pada penelitian ini (Subbab 4.2) memisahkan sepenuhnya kedua proses tersebut, sehingga *iteration cycle* pengembangan *firmware* jauh lebih cepat karena tidak memerlukan sintesis maupun pemrograman ulang FPGA setiap kali program berubah.

Menariknya, tim `onchipuis` sendiri turut mengalami kendala serupa dengan modul memori. Mereka juga menulis ulang modul RAM dari nol untuk port Vivado

mereka, menggunakan *template* Xilinx `xilinx_single_port_ram_write_first` (mode *write-first*). Pendekatan ini berbeda dari implementasi `SP32B1024.v` (Subbab 4.4.1) pada penelitian ini yang menggunakan mode *read-old-data*. Konvergensi independen ini menguatkan temuan bahwa modul memori memang tidak tersedia pada rilis *open-source* mriscv untuk target FPGA manapun, bukan keterbatasan khusus pada *open-source toolchain* yang dipakai penelitian ini.

BAB V
TAMBAHAN (OPSIONAL)

Anda boleh menambahkan Bab jika diperlukan. Jumlah Bab tidak harus sesuai dengan *template*.

Bab tambahan ini diperlukan jika hasil penelitian untuk menjawab tujuan cukup panjang atau terdiri dari banyak sub bab. Mahasiswa boleh menjawab 1 tujuan penelitian dengan 1 bab.

BAB VI
KESIMPULAN DAN SARAN

6.1 Kesimpulan

Kesimpulan dapat diawali dengan apa yang dilakukan dengan tugas akhir ini lalu dilanjutkan dengan poin-poin yang menjawab tujuan penelitian, apakah tujuan sudah tercapai atau belum, tentunya berdasarkan data ataupun hasil dari Bab pembahasan sebelumnya. Dalam beberapa hal, kesimpulan dapat juga berisi tentang temuan/*findings* yang Anda dapatkan setelah melakukan pengamatan dan atau analisis terhadap hasil penelitian.

Kesimpulan menjawab seberapa jauh rumusan masalah tercapai berdasarkan hasil penelitian. Semua rumusan masalah harus disimpulkan berdasarkan data penelitian.

6.2 Saran

Saran berisi hal-hal yang bisa dilanjutkan dari penelitian atau skripsi ini, yang belum dilakukan karena batasan permasalahan. Saran bukan berisi saran kepada sistem atau pengguna, tetapi saran diberikan kepada aspek penelitian yang dapat dikembangkan dan ditambahkan di penelitian atau skripsi selanjutnya.

Catatan: Mahasiswa perlu melihat sinkronisasi antara rumusan masalah, tujuan, metode, hasil penelitian, dan kesimpulan.

DAFTAR PUSTAKA

[1] A. Waterman, Y. Lee, D. Patterson, and K. Asanović, "The risc-v instruction set manual, volume i: User-level isa, version 2.1," EECS Department, University of California, Berkeley, Tech. Rep. UCB/EECS-2016-118, May 2016.

[2] S. L. Harris and D. Harris, *Digital Design and Computer Architecture: RISC-V Edition*. San Francisco, CA: Morgan Kaufmann, 2021.

[3] V. V. Solov'ev, "Asmd-fsmd technique in designing signal processing devices on field programmable gate arrays," *Journal of Communications Technology and Electronics*, vol. 66, no. 12, pp. 1336–1345, 2021.

[4] M. Darvishi, "Pipeline-stage-resolved timing characterization of fpga and asic implementations of a risc-v processor," *arXiv preprint arXiv:2512.13866*, 2025.

[5] D. Shah, E. Hung, C. Wolf, S. Bazanski, D. Gisselquist, and M. Milanovic, "Yosys+nextpnr: An open source framework from verilog to bitstream for commercial fpgas," in *Proceedings of the 2019 IEEE 27th Annual International Symposium on Field-Programmable Custom Computing Machines (FCCM)*. IEEE, 2019, pp. 1–4.

Catatan: Daftar pustaka adalah apa yang dirujuk atau disitasi, bukan apa yang telah dibaca, jika tidak ada dalam sitasi maka tidak perlu dituliskan dalam daftar pustaka.

LAMPIRAN

L.1 Isi Lampiran

Lampiran bersifat opsional bergantung hasil kesepakatan dengan pembimbing dapat berupa:

1. Bukti pelaksanaan Kuesioner seperti pertanyaan kuesioner, resume jawaban responden, dan dokumentasi kuesioner.
2. Spesifikasi Aplikasi atau Sistem yang dikembangkan meliputi spesifikasi teknis aplikasi, tautan unduh aplikasi, manual penggunaan aplikasi, hingga screenshot aplikasi.
3. Cuplikan kode yang sekiranya penting dan ditambahkan.
4. Tabel yang terlalu panjang yang masih diperlukan tetapi tidak memungkinkan untuk ditayangkan di bagian utama skripsi.
5. Gambar-gambar pendukung yang tidak terlalu penting untuk ditampilkan di bagian utama. Akan tetapi, mendukung argumentasi/pengamatan/analisis.
6. Penurunan rumus-rumus atau pembuktian suatu teorema yang terlalu panjang dan terlalu teknis sehingga Anda berasumsi bahwa pembaca biasa tidak akan menelaah lebih lanjut. Hal ini digunakan untuk memberikan kesempatan bagi pembaca tingkat lanjut untuk melihat proses penurunan rumus-rumus ini.

LAMPIRAN

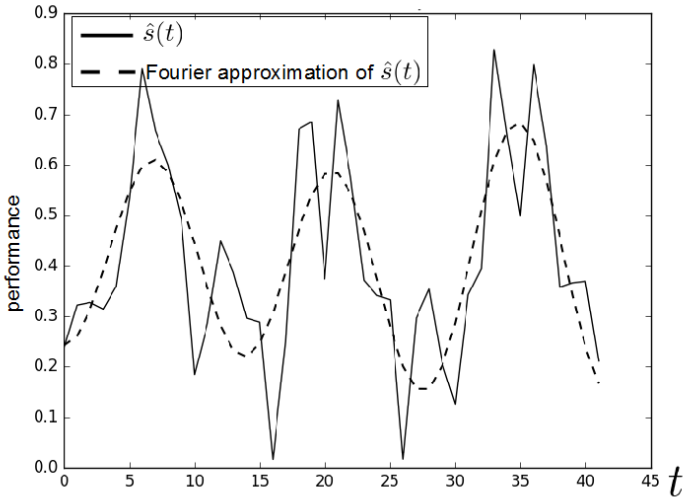
L.2 Panduan Latex

L.2.1 Syntax Dasar

L.2.1.1 Penggunaan Sitasi

Contoh penggunaan sitasi [**?**, **?**] [**?**] [**?**] [**?**] [**?**, **?**]

L.2.1.2 Penulisan Gambar



Gambar L.1. Contoh gambar.

Contoh gambar terlihat pada Gambar L.1. Gambar diambil dari [**?**].

L.2.1.3 Penulisan Tabel

Tabel L.1. Tabel ini

ID	Tinggi Badan (cm)	Berat Badan (kg)
A23	173	62
A25	185	78
A10	162	70

Contoh penulisan tabel bisa dilihat pada Tabel L.1.

L.2.1.4 Penulisan formula

Contoh penulisan formula

$$L_{\psi_z} = \{t_i \mid v_z(t_i) \leq \psi_z\} \tag{1}$$

Contoh penulisan secara *inline*: $PV = nRT$. Untuk kasus-kasus tertentu, kita membutuhkan perintah "mathit" dalam penulisan formula untuk menghindari adanya jeda saat penulisan formula.

Contoh formula **tanpa** menggunakan "mathit": $PVA = RTD$

Contoh formula **dengan** menggunakan "mathit": $PVA = RTD$

L.2.1.5 Contoh list

Berikut contoh penggunaan list

1. First item
2. Second item
3. Third item

L.2.2 Blok Beda Halaman

L.2.2.1 Membuat algoritma terpisah

Untuk membuat algoritma terpisah seperti pada contoh berikut, kita dapat memanfaatkan perintah *algstore* dan *algrestore* yang terdapat pada paket *algcompatible*. Pada dasarnya, kita membuat dua blok algoritma dimana blok pertama kita simpan menggunakan *algstore* dan kemudian di-restore menggunakan *algrestore* pada algoritma kedua. Perintah tersebut dimaksudkan agar terdapat kesinamungan antara kedua blok yang sejatinya adalah satu blok.

Algorithm 1 Contoh algorima

```
1: procedure CREATESET(v)
2:     Create new set containing v
3: end procedure
```

Pada blok algoritma kedua, tidak perlu ditambahkan caption dan label, karena sudah menjadi satu bagian dalam blok pertama. Pembagian algoritma menjadi dua bagian ini berguna jika kita ingin menjelaskan bagian-bagian dari sebuah algoritma, maupun untuk memisah algoritma panjang dalam beberapa halaman.

```
4: procedure CONCATSET(v)
5:     Create new set containing v
6: end procedure
```

L.2.2.2 Membuat tabel terpisah

Untuk membuat tabel panjang yang melebihi satu halaman, kita dapat mengganti kombinasi *table* + *tabular* menjadi *longtable* dengan contoh sebagai berikut.

Tabel L.2. Contoh tabel panjang

header 1	header 2
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar
foo	bar

L.2.2.3 Menulis formula terpisah halaman

Terkadang kita butuh untuk menuliskan rangkaian formula dalam jumlah besar sehingga melewati batas satu halaman. Solusi yang digunakan bisa saja dengan memindahkan satu blok formula tersebut pada halaman yang baru atau memisah rangkaian formula menjadi dua bagian untuk masing-masing halaman. Cara yang pertama mungkin akan menghasilkan alur yang berbeda karena ruang kosong pada halaman pertama akan diisi oleh teks selanjutnya. Sehingga di sini kita dapat memanfaatkan *align* yang sudah diatur dengan mode *allowdisplaybreaks*. Penggunakan *align* ini memungkinkan satu rangkaian formula terpisah berbeda halaman.

Contoh sederhana dapat digambarkan sebagai berikut.

$$\begin{aligned} x &= y^2 \\ x &= y^3 \\ a + b &= c \\ x &= y - 2 \\ a + b &= d + e \\ x^2 + 3 &= y \\ a(x) &= 2x \end{aligned} \tag{2}$$

$$b_i = 5x$$
$$10x^2 = 9x$$
$$2x^2 + 3x + 2 = 0$$
$$5x - 2 = 0$$
$$d = \log x$$
$$y = \sin x$$

LAMPIRAN

L.3 Format Penulisan Referensi

Penulisan referensi mengikuti aturan standar yang sudah ditentukan. Untuk internasionalisasi DTETI, maka penulisan referensi akan mengikuti standar yang ditetapkan oleh IEEE (*International Electronics and Electrical Engineers*). Aturan penulisan ini bisa diunduh di http://www.ieee.org/documents/ieeecitationref.pdf. Gunakan Mendeley sebagai *reference manager* dan *export* data ke format Bibtex untuk digunakan di Latex.

Berikut ini adalah sampel penulisan dalam format IEEE:

L.3.1 Book

Basic Format:

[1] J. K. Author, "Title of chapter in the book," in Title of His Published Book, xth ed. City of Publisher, Country: Abbrev. of Publisher, year, ch. x, sec. x, pp. xxx–xxx.

Examples:

[1] B. Klaus and P. Horn, Robot Vision. Cambridge, MA: MIT Press, 1986.

[2] L. Stein, "Random patterns," in Computers and You, J. S. Brake, Ed. New York: Wiley, 1994, pp. 55-70.

[3] R. L. Myer, "Parametric oscillators and nonlinear materials," in Nonlinear Optics, vol. 4, P. G. Harper and B. S. Wherret, Eds. San Francisco, CA: Academic, 1977, pp. 47-160.

[4] M. Abramowitz and I. A. Stegun, Eds., Handbook of Mathematical Functions (Applied Mathematics Series 55). Washington, DC: NBS, 1964, pp. 32-33.

[5] E. F. Moore, "Gedanken-experiments on sequential machines," in Automata Studies (Ann. of Mathematical Studies, no. 1), C. E. Shannon and J. McCarthy, Eds. Princeton, NJ: Princeton Univ. Press, 1965, pp. 129-153.

[6] Westinghouse Electric Corporation (Staff of Technology and Science, Aerospace Div.), Integrated Electronic Systems. Englewood Cliffs, NJ: Prentice-Hall, 1970.

[7] M. Gorkii, "Optimal design," Dokl. Akad. Nauk SSSR, vol. 12, pp. 111-122, 1961 (Transl.: in L. Pontryagin, Ed., The Mathematical Theory of Optimal Processes. New York: Interscience, 1962, ch. 2, sec. 3, pp. 127-135).

[8] G. O. Young, “Synthetic structure of industrial plastics,” in Plastics, vol. 3, Polymers of Hexadromicon, J. Peters, Ed., 2nd ed. New York: McGraw-Hill, 1964, pp. 15-64.

L.3.2 Handbook

Basic Format:

[1] Name of Manual/Handbook, x ed., Abbrev. Name of Co., City of Co., Abbrev. State, year, pp. xx-xx.

Examples:

[1] Transmission Systems for Communications, 3rd ed., Western Electric Co., Winston Salem, NC, 1985, pp. 44-60.

[2] Motorola Semiconductor Data Manual, Motorola Semiconductor Products Inc., Phoenix, AZ, 1989.

[3] RCA Receiving Tube Manual, Radio Corp. of America, Electronic Components and Devices, Harrison, NJ, Tech. Ser. RC-23, 1992.

Conference/Prosiding

Basic Format:

[1] J. K. Author, "Title of paper," in Unabbreviated Name of Conf., City of Conf., Abbrev. State (if given), year, pp.xxx-xxx.

Examples:

[1] J. K. Author [two authors: J. K. Author and A. N. Writer] [three or more authors: J. K. Author et al.], "Title of Article," in [Title of Conf. Record as], [copyright year] © [IEEE or applicable copyright holder of the Conference Record]. doi: [DOI number]

Sumber Online/Internet

Basic Format:

[1] J. K. Author. (year, month day). Title (edition) [Type of medium]. Available: http://www.(URL)

Examples:

[1] J. Jones. (1991, May 10). Networks (2nd ed.) [Online]. Available: http://www.atm.com

Skripsi, Tesis dan Disertasi

Basic Format:

[1] J. K. Author, "Title of thesis," M.S. thesis, Abbrev. Dept., Abbrev. Univ., City of Univ., Abbrev. State, year.

[2] J. K. Author, “Title of dissertation,” Ph.D. dissertation, Abbrev. Dept., Abbrev. Univ., City of Univ., Abbrev. State, year.

Examples:

[1] J. O. Williams, “Narrow-band analyzer,” Ph.D. dissertation, Dept. Elect. Eng., Harvard Univ., Cambridge, MA, 1993. [2] N. Kawasaki, “Parametric study of thermal and chemical nonequilibrium nozzle flow,” M.S. thesis, Dept. Electron. Eng., Osaka Univ., Osaka, Japan, 1993

LAMPIRAN

L.4 Contoh Source Code

L.4.1 Sample algorithm

Algorithm 2 Kruskal's Algorithm

```
1:  procedure MAKESET(v)
2:      Create new set containing v
3:  end procedure
4:
5:  function FINDSET(v)
6:      return a set containing v
7:  end function
8:
9:  procedure UNION(u,v)
10:     Unites the set that contain u and v into a new set
11: end procedure
12:
13: function KRUSKAL(V, E, w)
14:     A ← {}
15:     for each vertex v in V do
16:         MakeSet(v)
17:     end for
18:     Arrange E in increasing costs, ordered by w
19:     for each (u,v) taken from the sorted list do
20:         if FindSet(u) ≠ FindSet(v) then
21:             A ← A ∪ {(u, v)}
22:             Union(u, v)
23:         end if
24:     end for
25:     return A
26: end function
```

L.4.2 Sample Python code

```
import numpy as np

def incmatrix(genl1,genl2):
  m = len(genl1)
  n = len(genl2)
  M = None #to become the incidence matrix
  VT = np.zeros((n*m,1), int)  #dummy variable

  #compute the bitwise xor matrix
  M1 = bitxormatrix(genl1)
  M2 = np.triu(bitxormatrix(genl2),1)

  for i in range(m-1):
    for j in range(i+1, m):
      [r,c] = np.where(M2 == M1[i,j])
      for k in range(len(r)):
        VT[(i)*n + r[k]] = 1;
        VT[(i)*n + c[k]] = 1;
        VT[(j)*n + r[k]] = 1;
        VT[(j)*n + c[k]] = 1;

  if M is None:
    M = np.copy(VT)
  else:
    M = np.concatenate((M, VT), 1)

  VT = np.zeros((n*m,1), int)

  return M
```

L.4.3 Sample Matlab code

```
function X = BitXorMatrix(A,B)
%function to compute the sum without charge of two vectors

  %convert elements into usigned integers
  A = uint8(A);
  B = uint8(B);

  m1 = length(A);
  m2 = length(B);
  X = uint8(zeros(m1, m2));
  for n1=1:m1
    for n2=1:m2
      X(n1, n2) = bitxor(A(n1), B(n2));
    end
  end
```